

Requirements Engineering

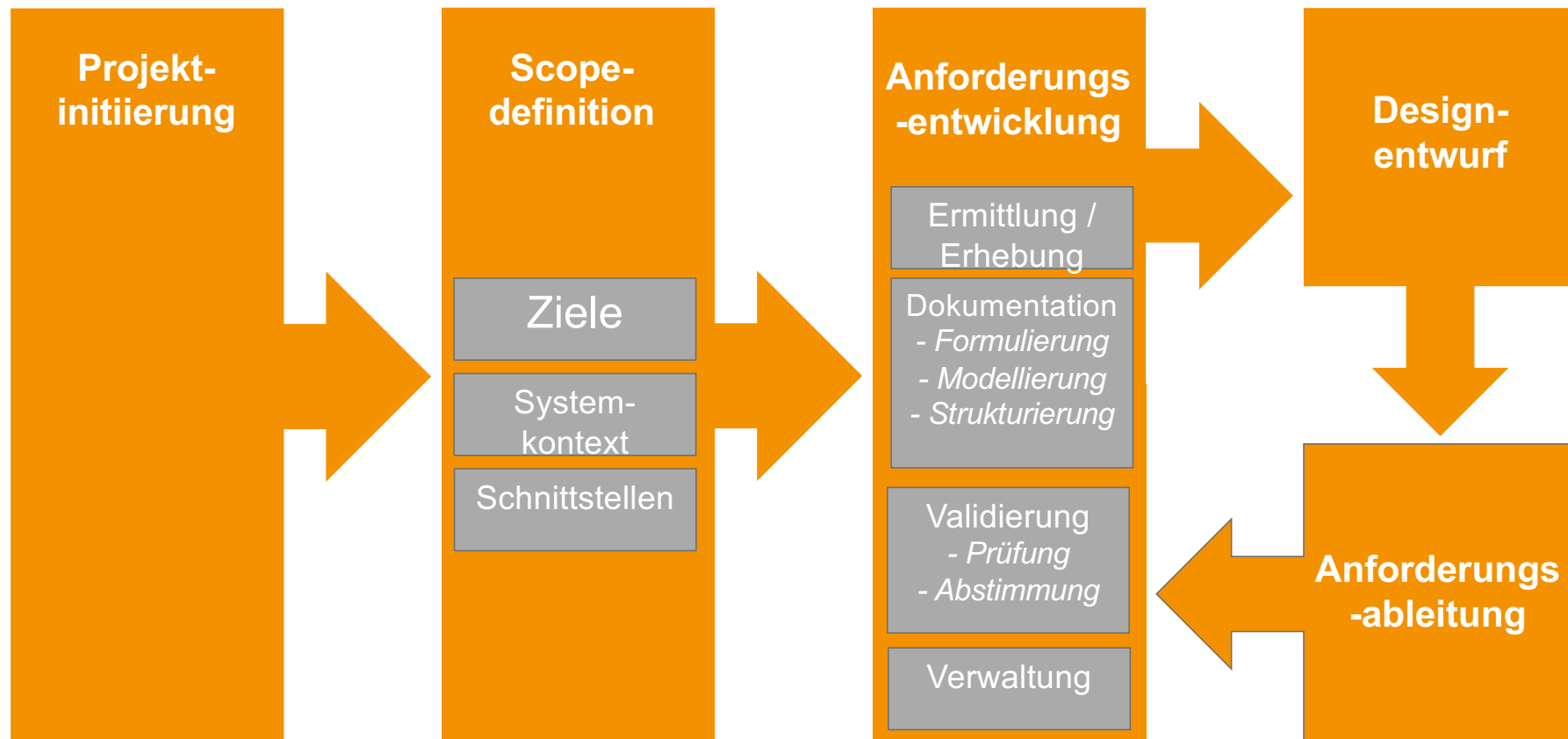
Requirements Engineering

- Klassisches Requirements Engineering

Vorgehen im RE

Vorgehen im RE

Klassisches Vorgehensmodell RE



Gruppenaufgabe

RE im V-Modell

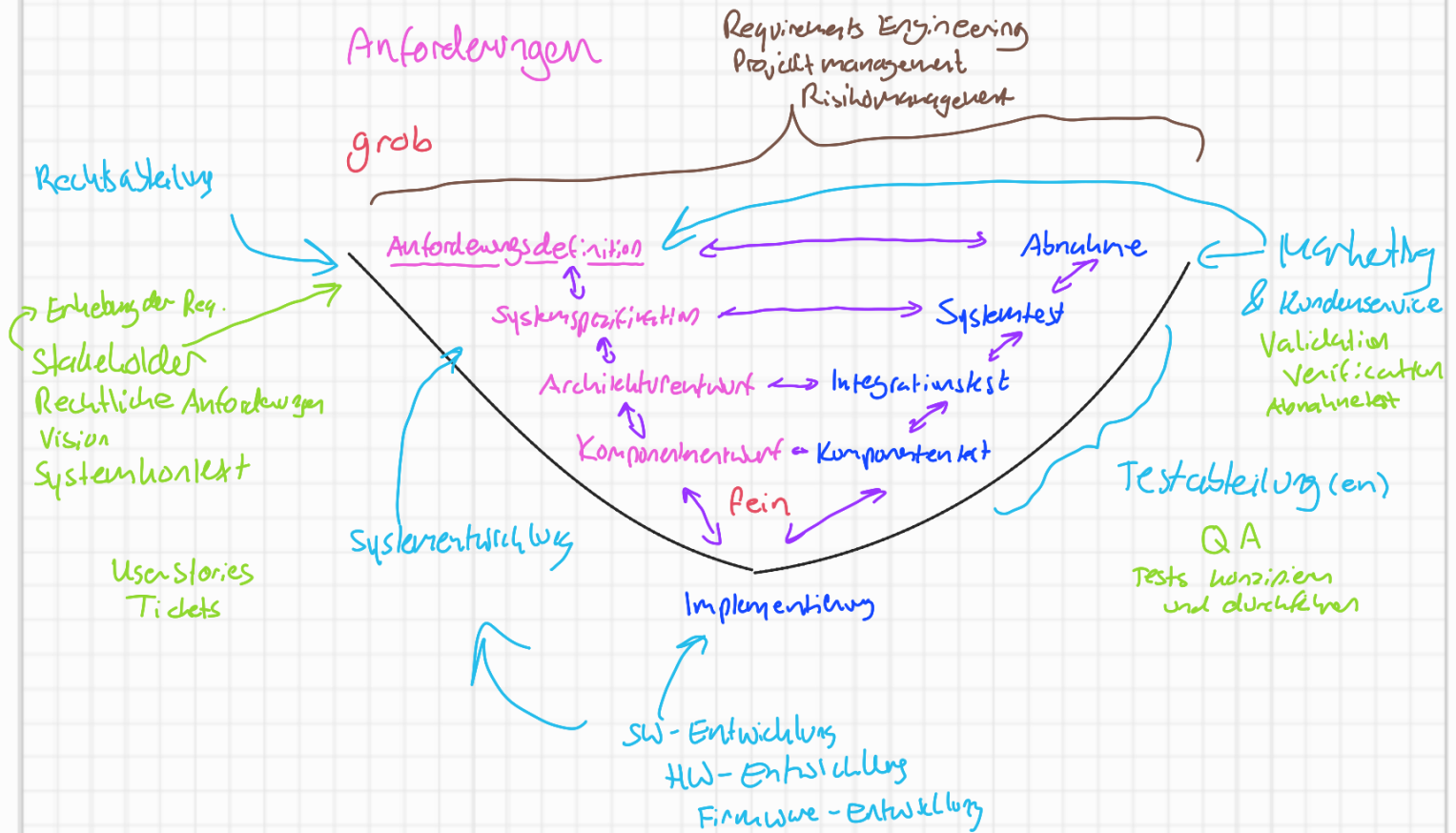
- Beschreiben sie graphisch, wie ihre Teams die RE-Aktivitäten durchführen sollen
- Skizzieren sie das V-Modell (als Basis für die Darstellung)
 - gemeinsame Besprechung
- Lokalisieren sie darin den RE-Prozess und die RE-Tätigkeiten.
- Ordnen sie beteiligte Teams bzw. Disziplinen den RE Tätigkeiten zu und kennzeichnen sie Beziehungen zwischen diesen.
- Zeitansatz: 20 Min

Vorgehen im RE

Man läuft durch diesen Zyklus durch

Aufgabe: Wo würden wir RE-Aktivitäten / Anforderungen im V-Modell modellieren?
Wie sollen die RE-Prozesse durchgeführt werden?
Welchen Phasen was zuordnen?

Kommunikation
Aufgaben
Teams



V-Modell XT – Einführung und Hintergrund

- **Definition und Ursprung**

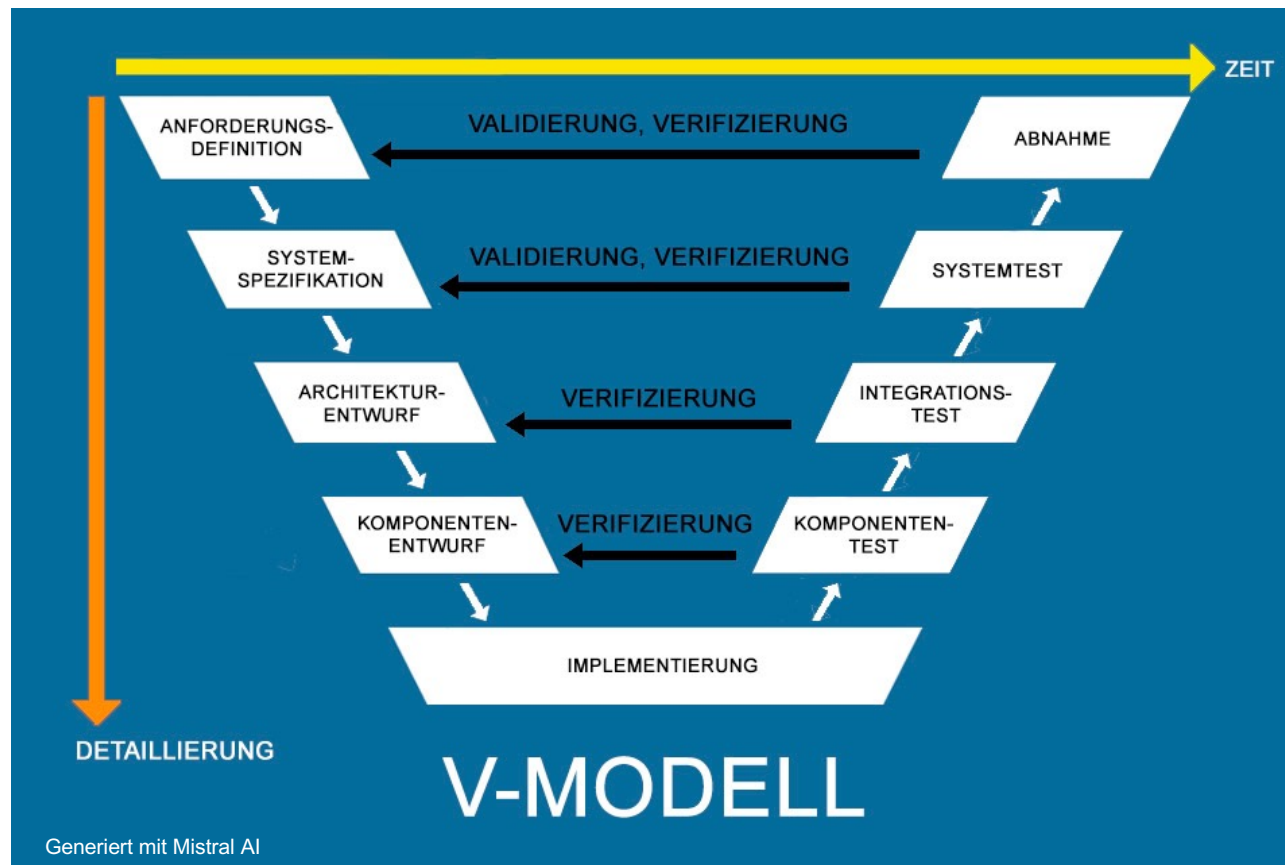
- Das V-Modell XT ist ein deutscher Standard zur Planung und Durchführung von Systementwicklungsprojekten, sowohl für Auftraggeber als auch Auftragnehmer. Es ist eine Weiterentwicklung des klassischen V-Modells und steht für „Extreme Tailoring“ – also die Möglichkeit, das Modell individuell an die Anforderungen eines Projekts anzupassen.
- Ursprünglich für IT-Projekte des Bundes entwickelt, wird es heute in vielen Branchen eingesetzt, insbesondere bei sicherheitskritischen oder komplexen Projekten

- **Ziele**

- Strukturierte und transparente Projektabwicklung
- Risikominimierung durch frühe Testphasen
- Qualitätssicherung durch klare Vorgaben und Dokumentation
- Flexibilität durch Tailoring: Anpassung an Projektgröße, Komplexität und Anforderungen

Vorgehen im RE

V-Modell XT



Vorgehen im RE

Beschreibung V-Modell XT

- **Phasenmodell**

- Das V-Modell XT besteht aus einem V-förmigen Ablauf: Auf der linken Seite stehen die Entwurfsphasen (z. B. Anforderungsanalyse, Systementwurf), auf der rechten Seite die korrespondierenden Testphasen (z. B. Systemtest, Abnahmetest).
- Jede Entwurfsphase hat eine direkte Testphase als Gegenstück, was die Qualitätssicherung bereits während der Entwicklung

- **Tailoring**

- Extreme Tailoring ermöglicht es, unnötige Phasen oder Aktivitäten zu streichen oder anzupassen, um den Aufwand für kleinere Projekte zu reduzieren.
- Das Modell kann auch agile Methoden integrieren, um Flexibilität und Iteration zu ermöglichen

Vorgehen im RE

Vergleich V-Modell & V-Modell XT - I

Kriterium	V-Modell	V-Modell XT
Entstehung	Ursprünglich in den 1990er Jahren als Standard für IT-Projekte des Bundes entwickelt.	Weiterentwicklung des klassischen V-Modells, eingeführt ab 2005.
Zweck	Strukturierte, lineare Vorgehensweise für Systementwicklungsprojekte.	Flexibler Standard mit Fokus auf Anpassbarkeit („Extreme Tailoring“) für verschiedene Projektgrößen und -arten.

Vorgehen im RE

Vergleich V-Modell & V-Modell XT - II

Kriterium	V-Modell	V-Modell XT
Phasenaufbau	V-förmiger Ablauf: Entwurfsphasen (links) und korrespondierende Testphasen (rechts).	Gleicher V-förmiger Aufbau, aber mit Möglichkeit, Phasen zu streichen oder anzupassen.
Flexibilität	Starre Struktur, wenig Anpassungsmöglichkeiten.	Extreme Tailoring: Phasen und Aktivitäten können je nach Projektanforderungen angepasst oder weggelassen werden.
Agile Integration	Kaum möglich, da linear und nicht iterativ.	Ermöglicht die Integration agiler Methoden, z. B. durch iterative Tailoring-Optionen.

Vorgehen im RE

Vergleich V-Modell & V-Modell XT - III

Kriterium	V-Modell	V-Modell XT
Eignung	Besonders für große, klar definierte Projekte mit hohem Dokumentationsbedarf.	Geeignet für Projekte jeder Größe, auch für kleinere oder agile Vorhaben durch Tailoring.
Branchen	Ursprünglich für öffentliche IT-Projekte, heute auch in der Industrie.	Breiter einsetzbar, z. B. in sicherheitskritischen, komplexen oder innovativen Projekten.

Vorgehen im RE

Vorteile V-Modell & V-Modell XT

Kriterium	V-Modell	V-Modell XT
Struktur	Klare, nachvollziehbare Projektstruktur.	Beibehaltung der klaren Struktur, aber mit mehr Flexibilität.
Qualitätssicherung	Frühe Testphasen reduzieren Risiken.	Gleicher Vorteil, zusätzlich durch Tailoring optimierbar.
Dokumentation	Umfassende Dokumentation für Transparenz und Nachvollziehbarkeit.	Dokumentation bleibt wichtig, kann aber an Projektgröße angepasst werden.

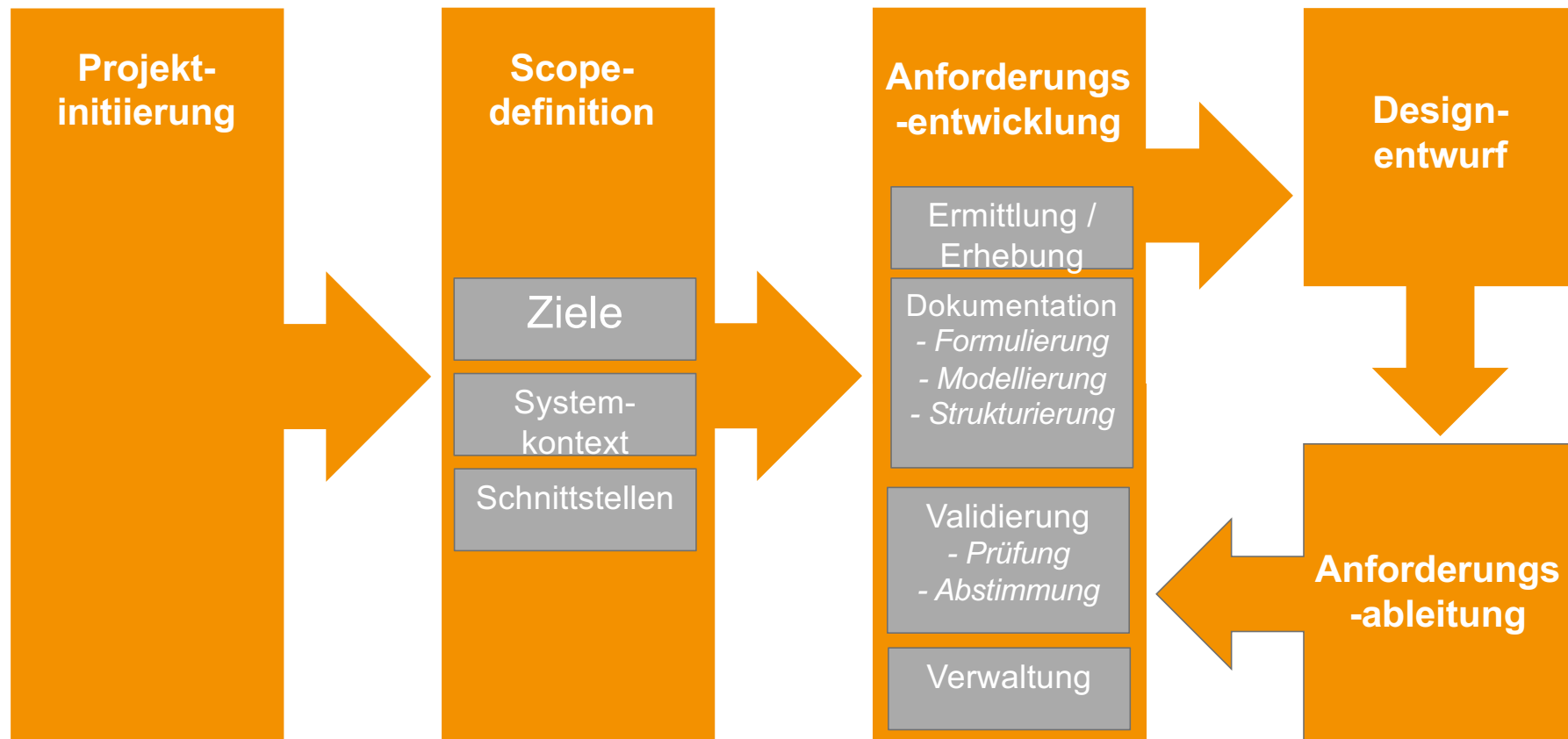
Vorgehen im RE

Nachteile V-Modell & V-Modell XT

Kriterium	V-Modell	V-Modell XT
Rigidität	Starre Phasenabfolge kann bei dynamischen Projekten hinderlich sein.	Tailoring erfordert Erfahrung; falsche Anpassungen können Qualitätsverlust bedeuten.
Dokumentationsaufwand	Hoher Aufwand, besonders bei großen Projekten.	Dokumentationsaufwand bleibt hoch, kann aber durch Tailoring reduziert werden.
Agilität	Kaum iterativ, schwer mit agilen Methoden kombinierbar.	Agile Elemente integrierbar, aber kein reines agiles Modell.

Vorgehen im RE

Klassisches Vorgehensmodell RE



Vorgehen im RE

Wann setzt man auf klassisches RE, wann auf agiles RE?

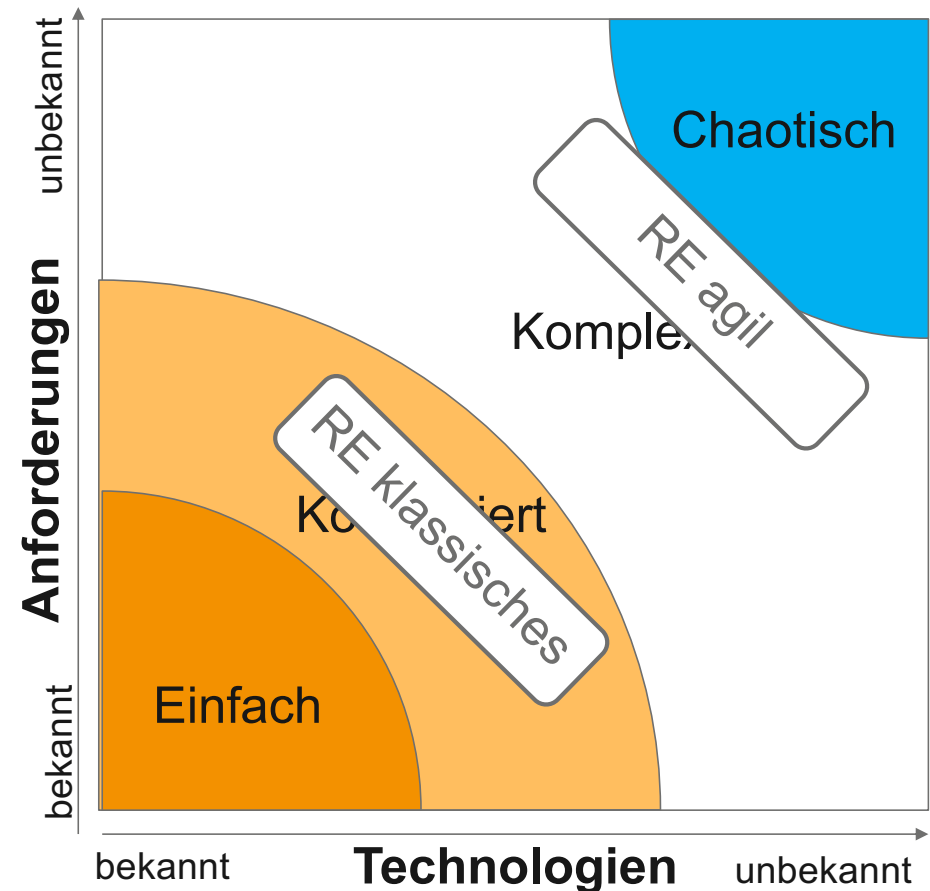
Stacy Matrix

In welche Kategorie lässt sich das Problem/die Initiative/die Aufgabe einordnen?

- i. Einfach
- ii. Kompliziert
- iii. Komplex
- iv. Chaotisch

➤ Wie bekannt ist das **WAS (Anforderungen)**?

➤ Wie bekannt ist das **WIE (Technologie)**?



Modellierung von Anforderungen

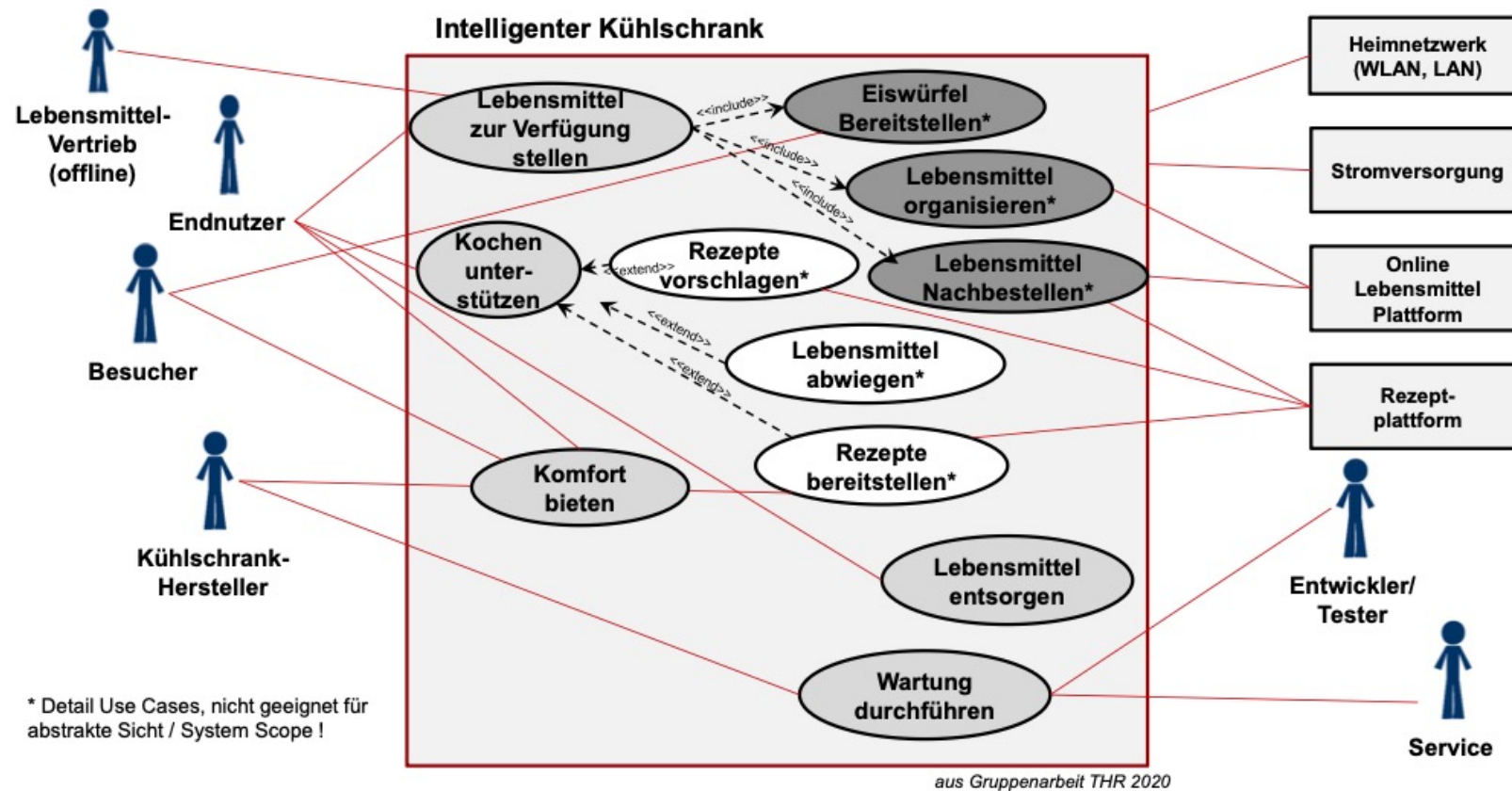
Modellierung von Anforderungen

Spezifikation eines Use Cases

- Use Case Spezifikation detailliert einen Use Case (z.B. aus dem Use Case Diagramm)
- Use Case-Spezifikationen enthalten typischerweise folgende Informationen:
 - Eindeutiger Bezeichner des Use Cases
 - Name des Use Cases
 - Kurzbeschreibung des Use Cases
 - Auslösendes Ereignis
 - Akteure
 - Ergebnis
 - Vor- und Nachbedingungen
 - Abläufe des Use Cases. Da es oft mehrere alternative Ablaufmöglichkeiten gibt, wird meist ein sogenannter „Hauptablauf“ und entsprechende „Alternative Abläufe“ beschrieben. Diese Abläufe nennt man auch Szenarien.

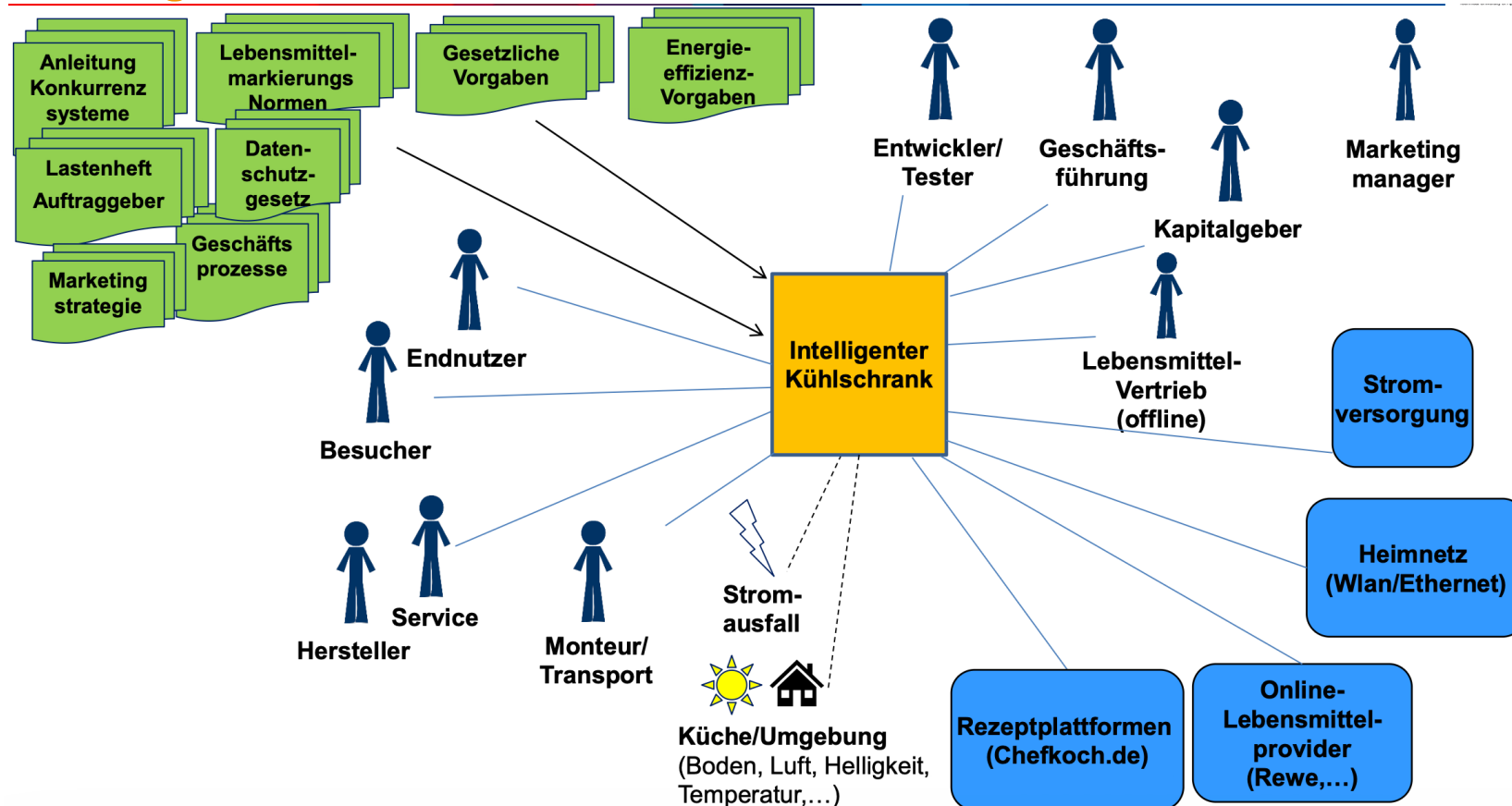
Modellierung von Anforderungen

Use Case Diagramm



Modellierung von Anforderungen

Kontextdiagramm



Modellierung von Anforderungen

Detailierungslevel von Use Cases

Detailierungslevel	Zweck	unterstützt
Grob beschrieben	Identifikation des Use Case und kurze Beschreibung seines Zwecks	• Scopemanagement • Diskussionen über Anforderungen
Übersicht mit Aufzählungszeichen	Beschreibung des wesentlichen (fachlichen) Ablaufs anhand des Basic Flows und Identifikation von Alternative Flows	• Scopemanagement • Grobe Schätzungen • Testdefinition im Team • Auswirkungsanalyse und Prototyping • Identifikation von Komponenten
Wesentliche Übersicht	Erweiterung des Basic Flow um Actor-System-Interaktionen	• UI-Design • Prototyping • Analyse und Design im Team • Testdefinitionen im Team • Genauere Schätzungen
Vollständig beschrieben	Erzeuge eine detaillierte Anforderungsspezifikation für den Use Case	• Analyse und Design • Implementierung und Testen • Erstellung der Benutzerdokumentation • Genauere Schätzung

Modellierung von Anforderungen

Template für die Use Case Spezifikation

Use Case ID		Use Case Name	
Created By		Last Updated By	
Date Created		Date Last Update	
Description			
Actor			
Priority		Frequency of Use	
Precondition			
Trigger			
Normal Course			
Alternative Course(s)			
Result			
Postcondition			
Special Requirements			
Assumptions			
Notes and Issue			

Modellierung von Anforderungen

Gruppenaufgabe: Use-Case Spezifikation

- Erstellen sie eine Use Case Spezifikation für einen der Use Cases für unseren smarten Mähdrescher.
- Benutzen Sie die Detailtiefe „Aufzählung“ und beschreiben Sie den den Basic Flow (mit ca. 5 – 10 Schritten)
- Löschen Sie die vorerst nicht benötigten Felder des Templates

- Zeitansatz: 15 Minuten

Modellierung von Anforderungen

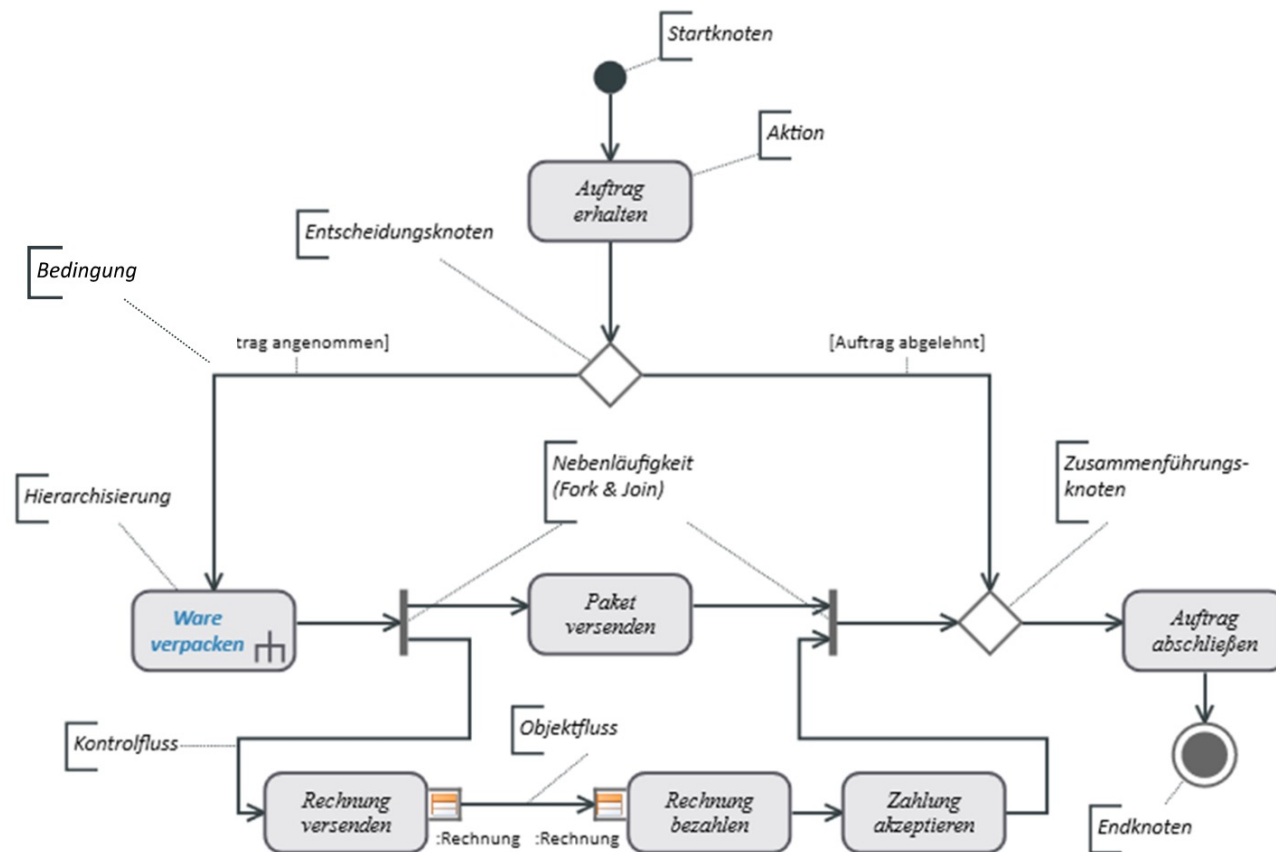
UML-Aktivitätsdiagramme

Aktivitätsdiagramm zur Darstellung von:

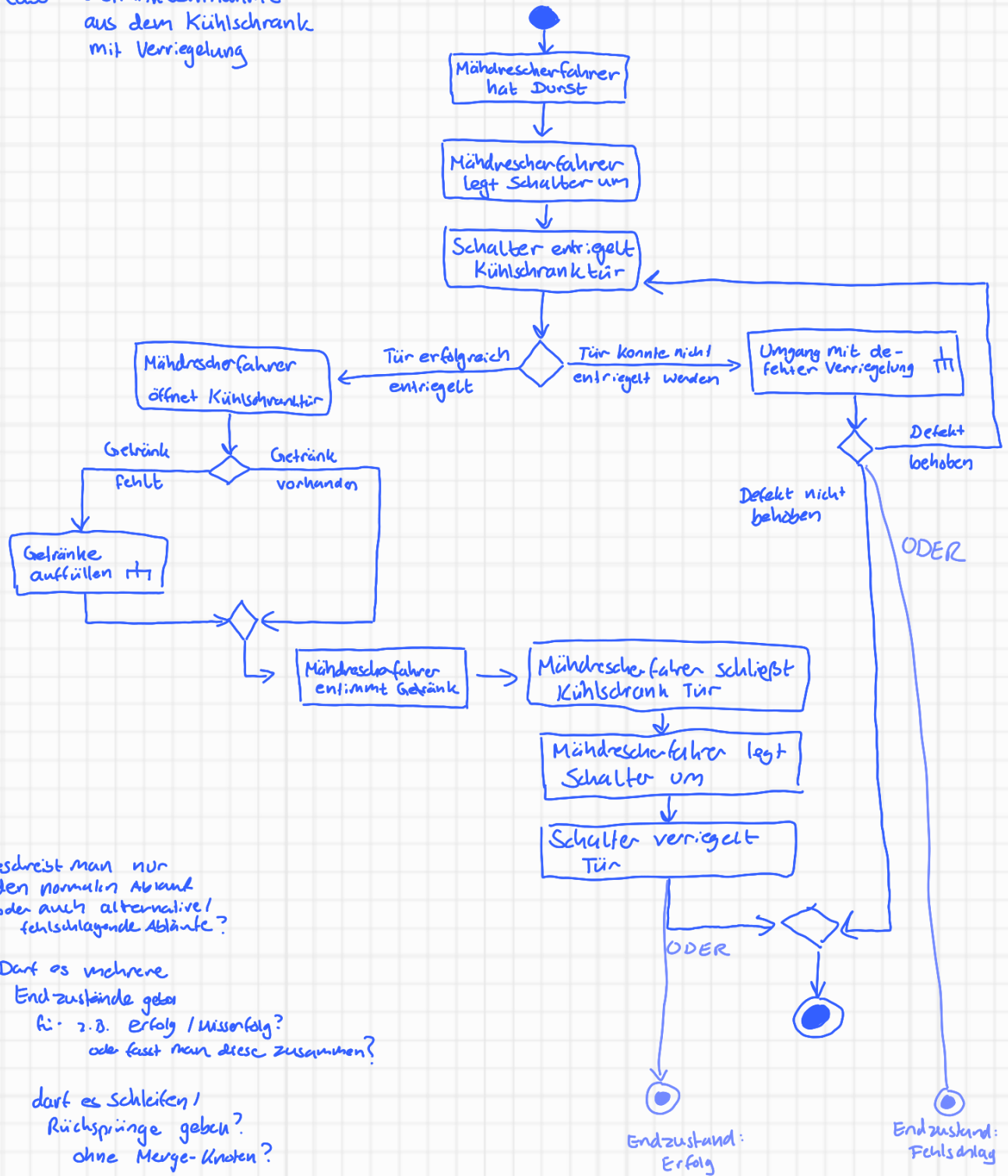
- detaillierten Use Case Abläufen
- Use Case Spezifikationen
- Ablauf von Geschäftsprozessen

Modellierung von Anforderungen

Aktivitätsdiagramm: Notationselemente



Use Case: Getränkeentnahme aus dem Kühlschrank mit Verriegelung



Beschreibt man nur den normalen Ablauf oder auch alternative / fehlschlagende Abläufe?

Darf es mehrere Endzustände geben?
 hi: z.B. Erfolg / Misserfolg?
 oder fasst man diese zusammen?

darf es Schleifen / Rücksprünge geben?
 ohne Merge-Knoten?



Modellierung von Anforderungen

Gruppenaufgabe: Use-Case Spezifikation

- Modellieren Sie den Ablauf eines Use Cases mithilfe eines Aktivitätsdiagramms.
 - Sie können dazu auch die zuvor erstellte Beschreibung aus der Use Case Spezifikation modellieren.
-
- Zeitansatz: 20 Minuten

Einschub: Modelle und Modellierung von Zielen

Modellierung von Anforderungen

Warum Modelle?

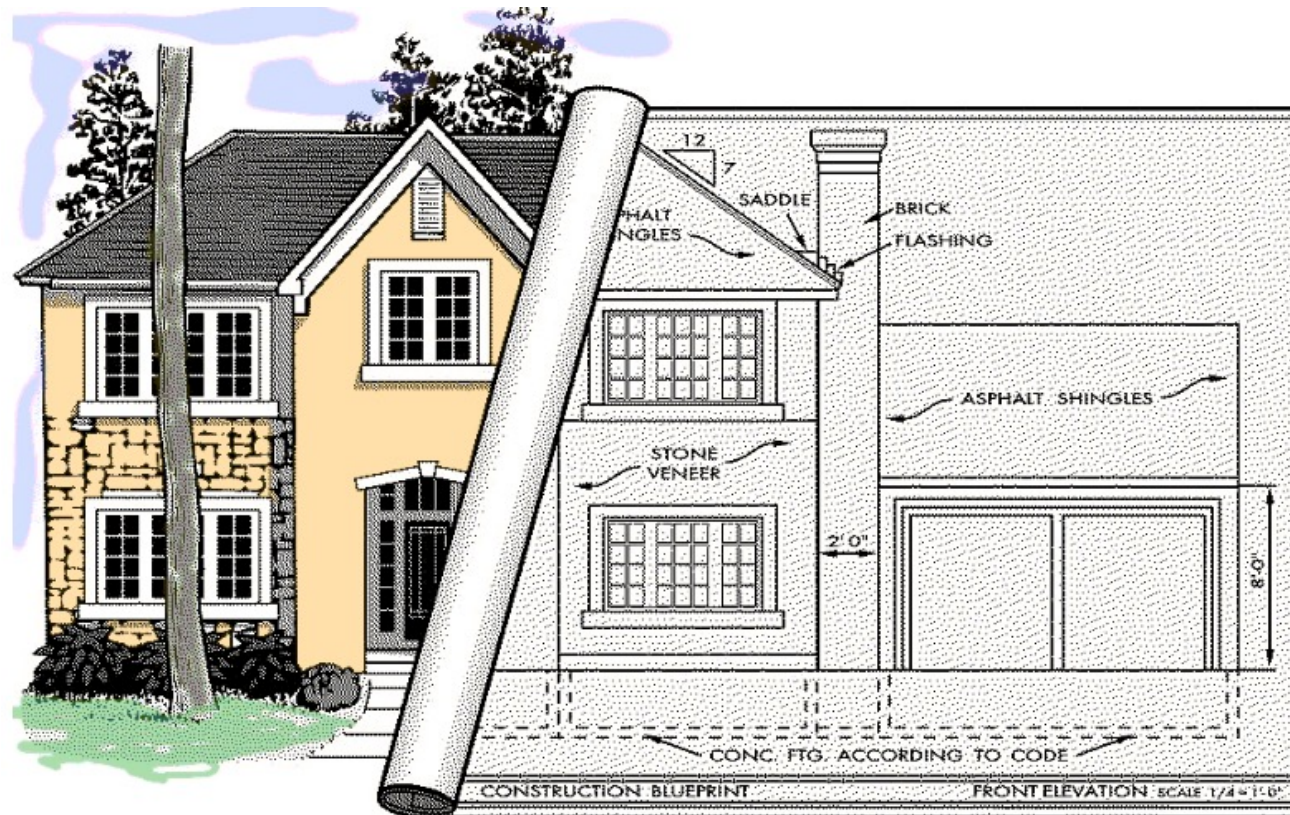
- Wir haben in der Vorlesung nun bereits zwei Diagramme kennengelernt (Use Case Diagramme und Aktivitätsdiagramme).



Bildquelle: <https://www.instructables.com/Scale-model-house/>

Modellierung von Anforderungen

Warum Modelle?



Modellierung von Anforderungen

Definition Modell

- Definition: Ein Modell ist ein abstrahierendes Abbild der Realität oder der zu schaffenden Realität
- Modelle besitzen drei zentrale Charakteristika:
 - *Abbildungseigenschaft*: Modelle bilden eine (zu schaffende) Realität ab
 - *Verkürzende Eigenschaft*: Modelle verkürzen die abgebildete Realität
 - *Pragmatische Eigenschaft*: Modelle werden für einen spezifischen Verwendungszweck konstruiert

Modellierung von Anforderungen

Vorteile von Modellen

- Im Vergleich zur Dokumentation in natürlicher Sprache bieten Anforderungsmodelle unter anderem folgende Vorteile:
 - **Grafisch dargestellte Informationen** können **schneller erfasst** und besser im **Gedächtnis behalten werden**
 - Mit unterschiedlichen Diagrammen können gezielt **bestimmte Perspektiven auf Anforderungen** modelliert werden
 - Durch die Definition einer für den jeweiligen Verwendungszweck besonders gut geeigneten Modellierungssprache können **Abstraktionen der Realität festgelegt werden**, die jeweils spezielle Zwecke unterstützen (z.B. haben Wanderkarten, Autobahnkarten, Stadtpläne, U-Bahn-Pläne unterschiedliche Darstellungssprachen und jeweils einen anderen Zweck; aber alle bilden die gleiche Realität ab)

Modellierung von Anforderungen

Vorteile von Modellen

- Modellierungssprachen (wie z.B. UML, Entity-Relationship-Modelle, etc.) werden durch zwei Aspekte definiert:
 - **Syntax:** Welche Modellierungselemente gibt es und wie dürfen diese miteinander kombiniert werden
 - **Semantik:** Welche Bedeutung haben die Modellierungselemente

Modellierung von Anforderungen

Unterschied Modell vs. Diagramm

- Nicht immer wird genau zwischen Modell und Diagramm unterschieden. Hilfreich ist folgende Definition:
 - **Modell:** Modelle ist die abstrakte Sicht auf ein System. Alle Teil-Modelle zusammen beschreiben logisch das System und bilden **sozusagen die „Datenbank“**.
 - **Diagramm:** Diagramme sind konkrete (Teil-)Darstellungen des Systems, oder Sichtweisen auf die „Datenbank“.

Modellierung von Anforderungen

Drei Perspektiven für die Modellierung

Die Anforderungen an ein zu entwickelndes System werden häufig in Form von drei Perspektiven beschrieben.

Für jede dieser Perspektiven gibt es geeignete Modellierungsarten:

Strukturperspektive

- Darstellung einer physikalischen, logischen oder funktionalen Struktur des Systems
- Darstellung von Ein- und Ausgabedaten, sowie von Beziehungen zwischen Daten im System z.B. Entity-Relationship-Diagramm, UML-Klassendiagramm

Funktionsperspektive

- Darstellung der Funktionen im System
- Darstellung von Abläufen z.B. Datenflussdiagramm, UML-Aktivitätsdiagramm

Verhaltensperspektive

- Darstellung von System-Zuständen, Zustandswechseln,
- Darstellung von Reaktionen des Systems auf Ereignisse im Systemkontext z.B. Statechart, UML-Zustandsdiagramm

Modellierung von Anforderungen

Modellierung von Zielen von Initiativen

Ziel = beschreibt einen Wunsch oder Willen eines Stakeholders, der sich typischerweise auf charakteristische Merkmale des zu entwickelnden Systems bezieht.

Ziele eignen sich insbesondere dazu, die Vision des Systems zu verfeinern.

Die Verfeinerung von übergeordneten zu untergeordneten Zielen wird als Zieldekomposition bezeichnet.

Ziele und die Beziehungen zwischen Zielen können in verschiedenen Notationen formuliert sein

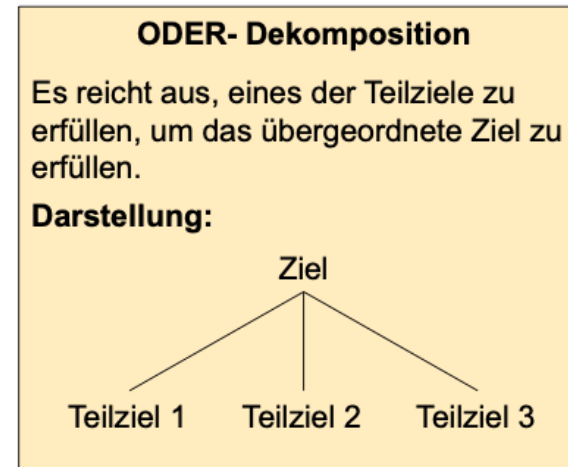
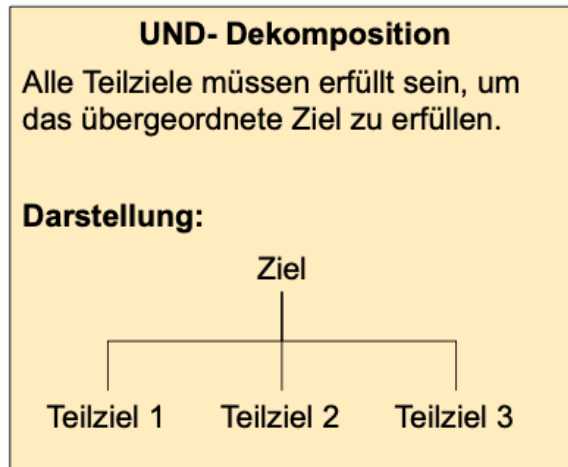
Modellierung von Anforderungen

Modellierung von Zielen von Initiativen

trägt zur Kosteneinschätzung bei und Herausfinden von must/nice-to-have
-> minimum viable product

- Ein Zielmodell sollte in der Lage sein, Dekompositionsbeziehungen zwischen Zielen (d.h. Beziehungen zwischen Ziel und den untergeordneten Teilzielen) zu beschreiben
- Die zwei Arten von Dekompositionsbeziehungen werden durch unterschiedliche Verbindungslinien dargestellt:

SMART Ziele
erreichbar und klar strukturiert



© Technische Hochschule Rosenheim, Prof. Dr. Christopher Jud, 30. April 2026, Seite 35, die Vorlesung basiert auf Material von Herrn Dr.-Ing. Thaddäus Dorsch

Bsp.: Prototypbau bis Deadline, Vorstellung auf Messe, etc als übergreifendes Ziel
untergeordnetes Ziel wäre dann der smarte Mähdrescher, dann weiter untergeordnet was den Mähdrescher ausmacht (UND) (z.b. autonomes Fahren, Ernten, Fahrer-Komfort, Analytics)
-> grundlegende Features, die das Produkt haben muss

grob definiert, dann weiter runtergebrochen in mögliche Implementierungen (ODER), die diese Ziele erfüllen könnten (z.b. Fahrer-Komfort > Kühlschrank/Klimaanlage)

Modellierung von Anforderungen

Gruppenaufgabe

- Modellieren sie Ziele für unseren smarten Mähdrescher in einem UND-ODER-Baum.
- Zeiteinsatz: 10 Min

UML-Diagramme

Modellierung von Anforderungen

UML-Klassendiagramme: Objekte und Klassen

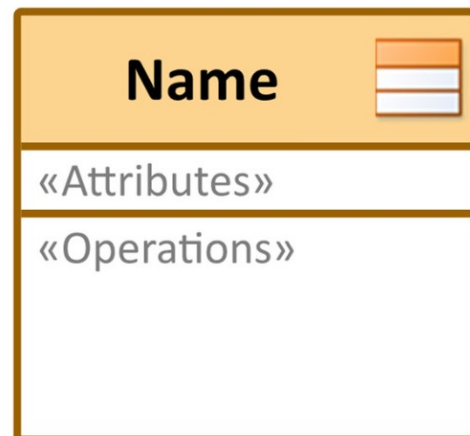
- Objekt: Ein Objekt kann jedes Ding sein, das eine Identität, eine Struktur und ein Verhalten besitzt.
- Klasse: Eine Klasse definiert eine Familie von Objekten, die eine gemeinsame Bedeutung, Struktur und ein gemeinsames Verhalten haben.



Modellierung von Anforderungen

UML-Klassendiagramme: Notationen

Notation für Klassen



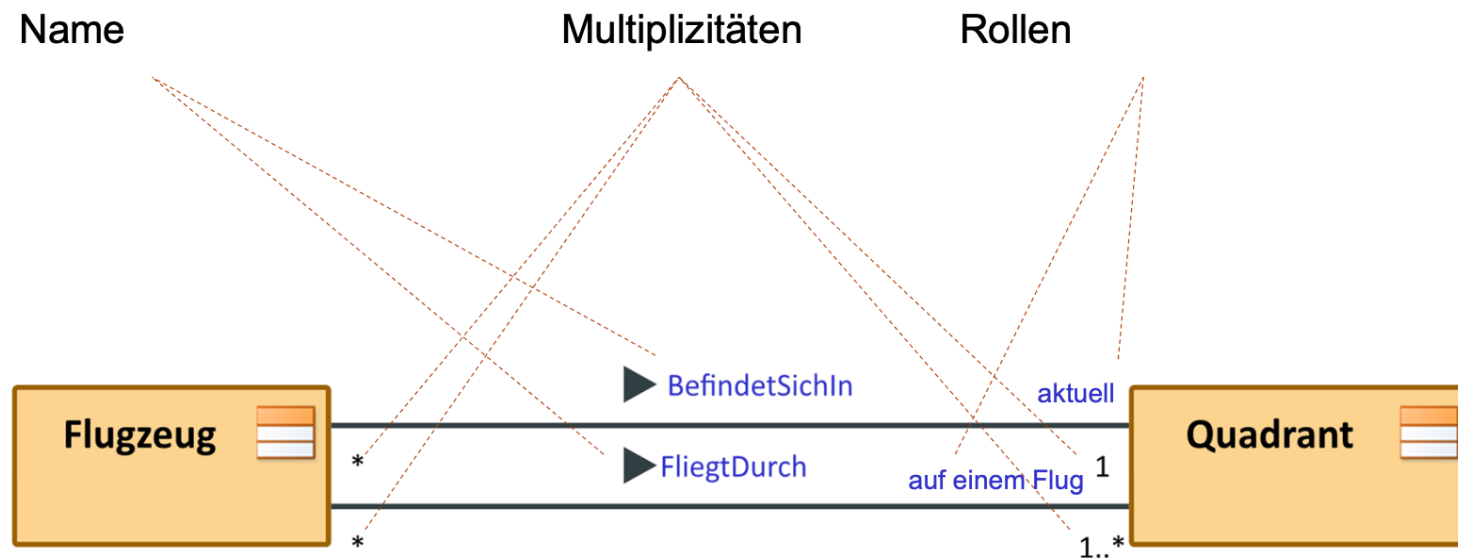
Beispiel für Klasse „Kunde“



Modellierung von Anforderungen

UML-Klassendiagramme: Assoziationen

- Assoziationen zeigen logische Beziehungen zwischen Klassen.
- Charakterisierung von Assoziationen durch:

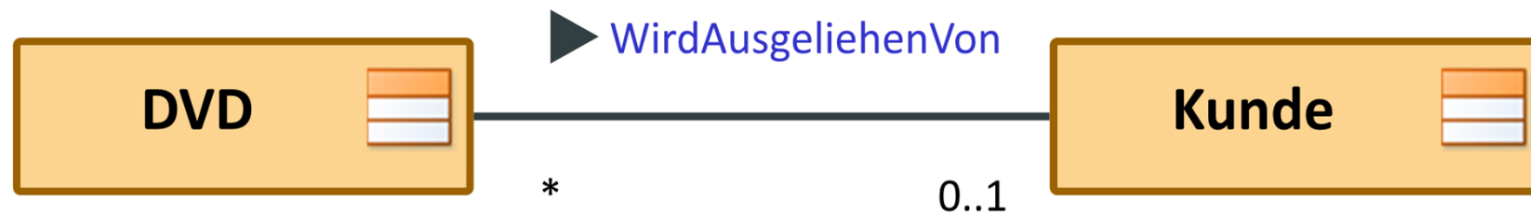


- Der Name kann entfallen, wenn es eine eindeutige Beziehung ist.

Modellierung von Anforderungen

UML-Klassendiagramme: Assoziationen und Multiplizitäten

- Zwei assoziierte Klassen:



Multiplizität:

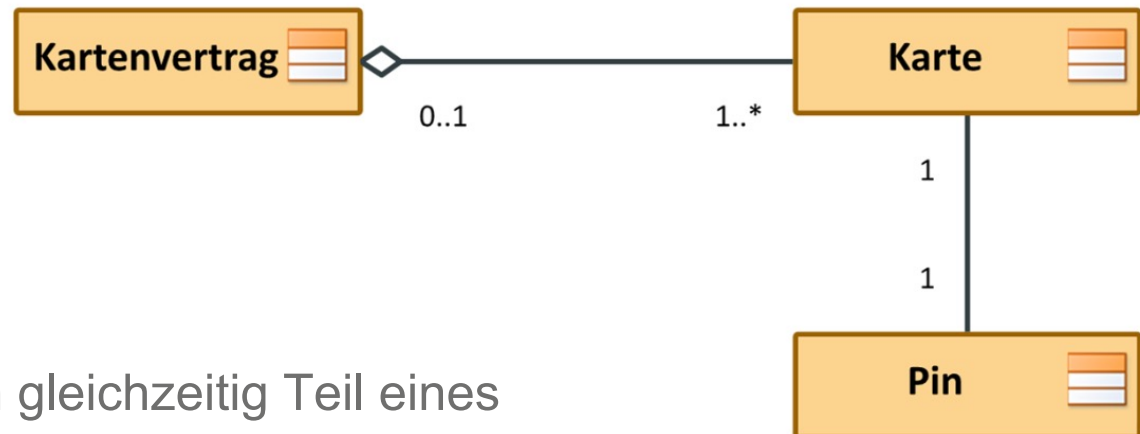
***** (=0..*): ein Objekt von Kunde kann 0 oder mehr Objekte der Klasse DVD ausleihen

0..1: ein Objekt von DVD kann durch kein oder maximal ein Objekt der Klasse Kunde ausgeliehen sein

Modellierung von Anforderungen

UML-Klassendiagramme: Assoziationen und Multiplizitäten

Aggregation („hat-ein“ – Beziehung)



Aggregation bedeutet:

- Eine Teil-Instanz (Karte) kann gleichzeitig Teil eines Kompositums (Kartenvertrag) sein.
- Entfällt das Kompositum, können seine Teile trotzdem weiter existieren (die Karte bleibt ohne Kartenvertrag bestehen).

Modellierung von Anforderungen

UML-Klassendiagramme: Assoziationen und Multiplizitäten

Komposition („ist Teil von“ – Beziehung)



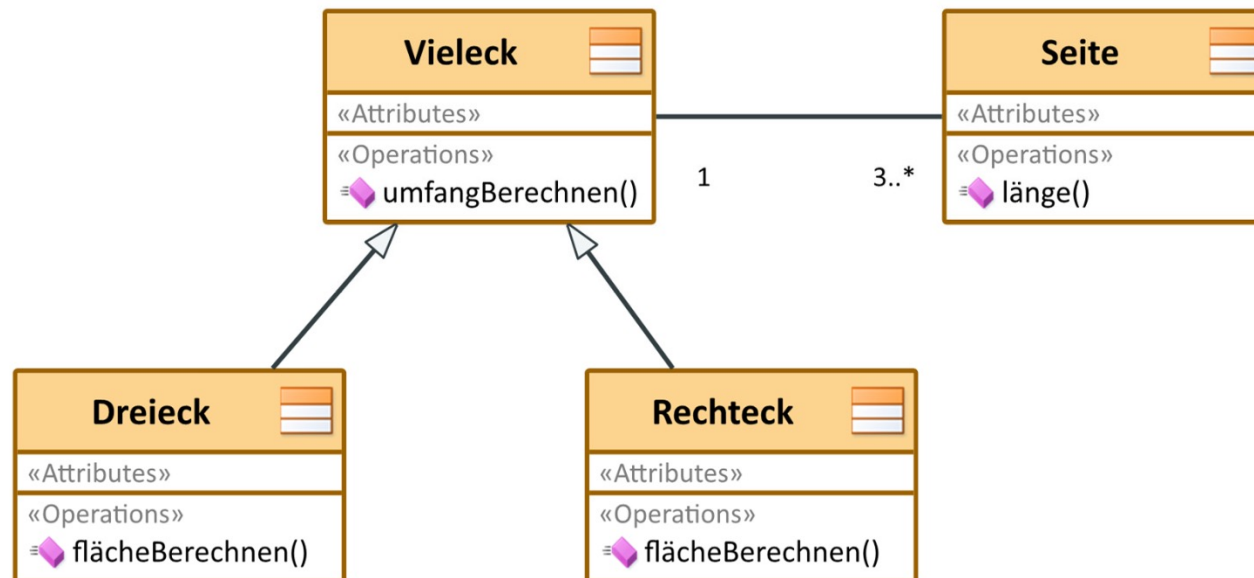
Komposition bedeutet:

- Eine Teil-Instanz (Verkaufsposition) kann gleichzeitig nur Teil eines Kompositums (Verkauf) sein.
- Eine Teil-Instanz muss immer zu einem Kompositum gehören.
- Das Kompositum ist allein für die Verwaltung seiner Teile, insbesondere für deren Erstellung und Zerstörung, verantwortlich.

Modellierung von Anforderungen

UML-Klassendiagramme: Assoziationen und Multiplizitäten

- Generalisierung (Vererbung, bzw. „ist ein“ – Beziehung)



- Eine Unterklasse (Dreieck, Rechteck) erbt von ihrer Oberklasse (Vieleck) die Attribute, Operationen und Assoziationen.

Modellierung von Anforderungen

Gruppenaufgabe: Klassendiagramm

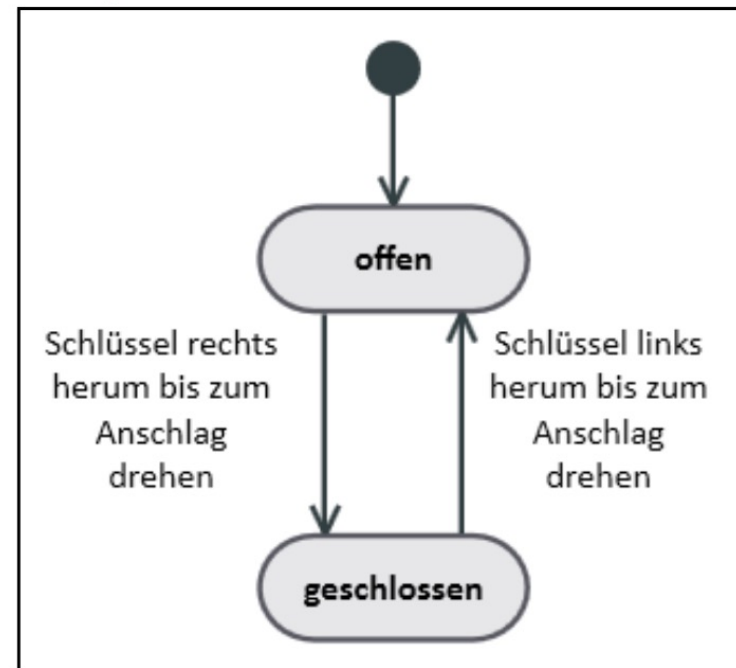
- Modellieren Sie die Dekomposition des Beispielsystems in Teilsysteme.
 - Alternativ wählen Sie ein spezielles Konzept oder eine Datenstruktur des Beispielsystems, das sie modellieren.
-
- Zeitansatz: 15 Minuten

Modellierung von Anforderungen

UML-Zustandsdiagramm

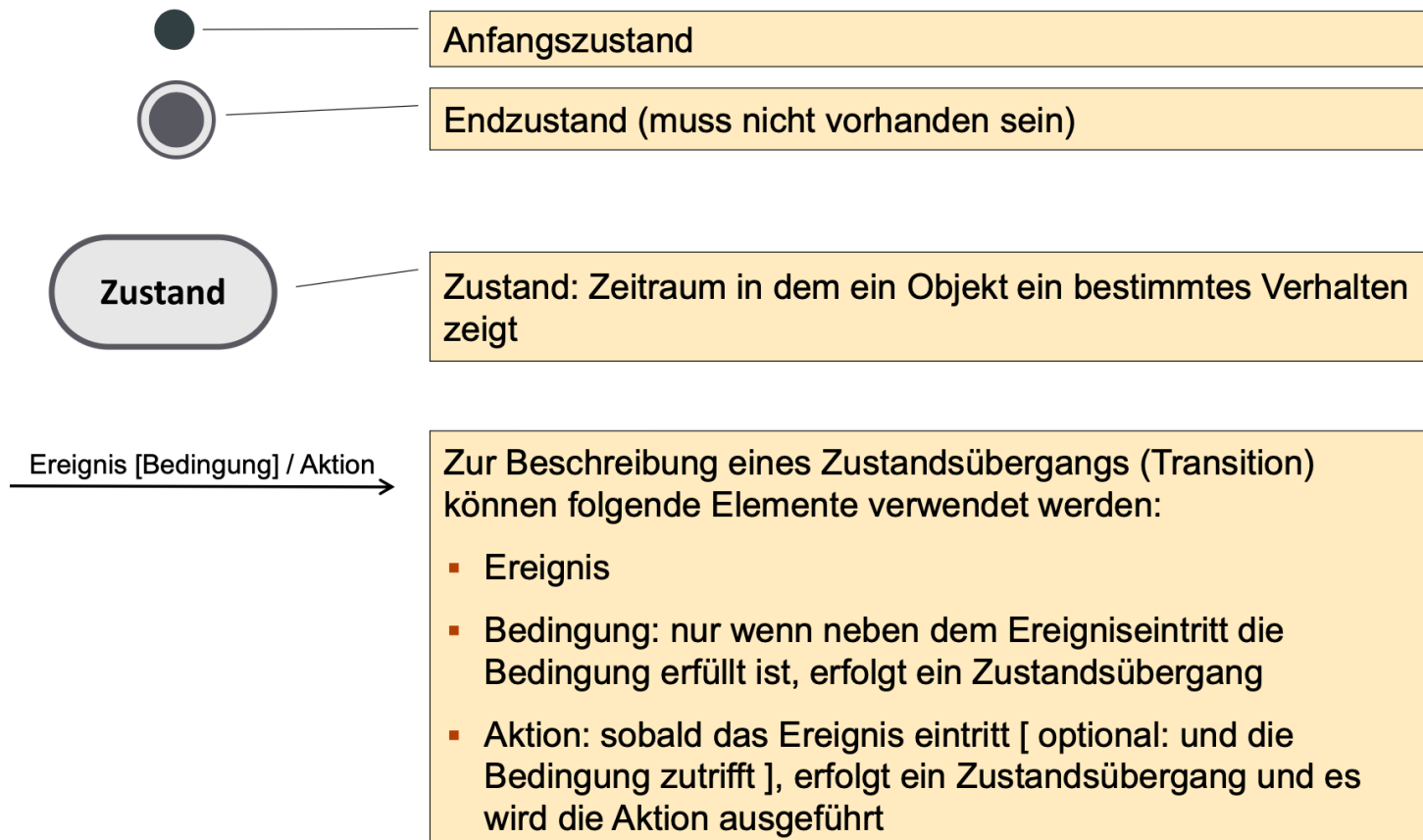
- Zeigt den Lebenszyklus eines Objektes im Laufe der Zeit.
- Ein Statechart oder Zustandsdiagramm zeigt die Reihenfolge der Verhaltensweisen eines Objektes.
- Ereignisse beeinflussen, welchen Zustand ein Objekt als nächstes einnimmt.
- Ein Ereignis benötigt keine Zeit.
- Die Zeit vergeht, während das Objekt sich in einem Zustand befindet.
- Statecharts und Zustandsdiagramme zeigen nicht, wie die Ereignisse oder Daten dem System zugeführt werden.

Beispiel: Zustände Türschloss



Modellierung von Anforderungen

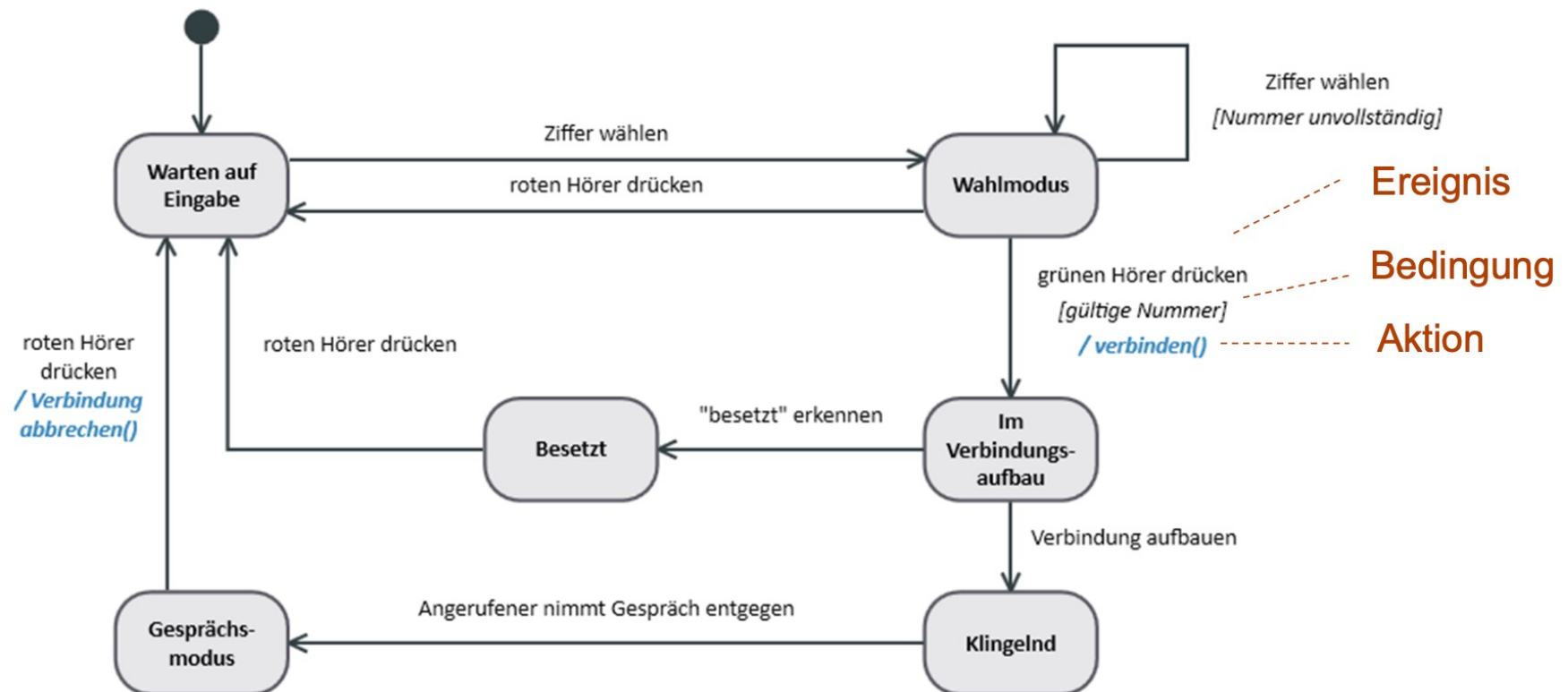
Zustandsdiagramm: Modellierungselemente



Modellierung von Anforderungen

Zustandsdiagramm: Beispiel

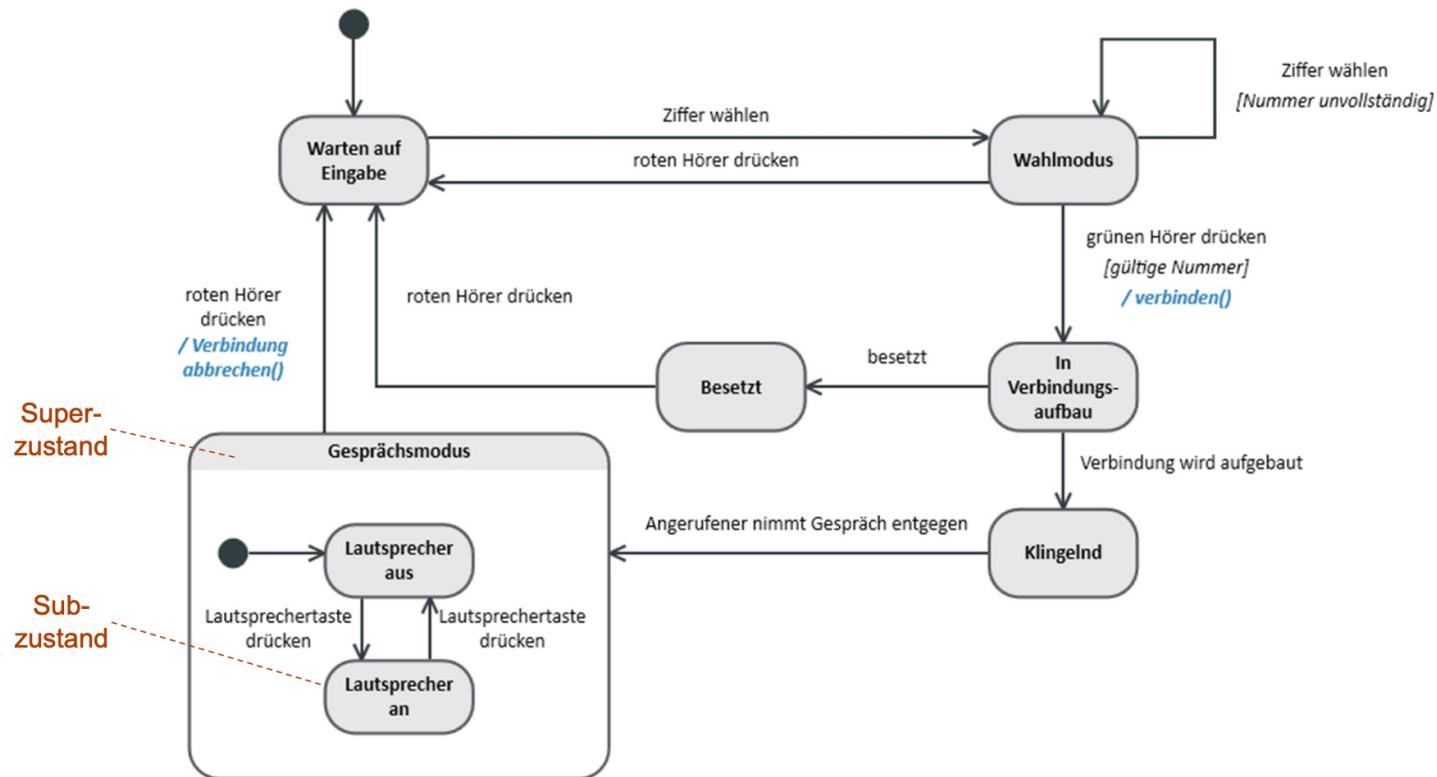
Modellierung des reaktiven Verhaltens eines Systems am Beispiel eines Telefons



Modellierung von Anforderungen

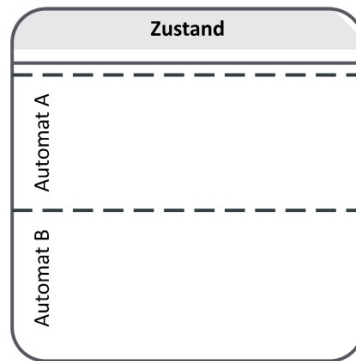
Zustandsdiagramm: Beispiel

Modellierung von Zustandshierarchien

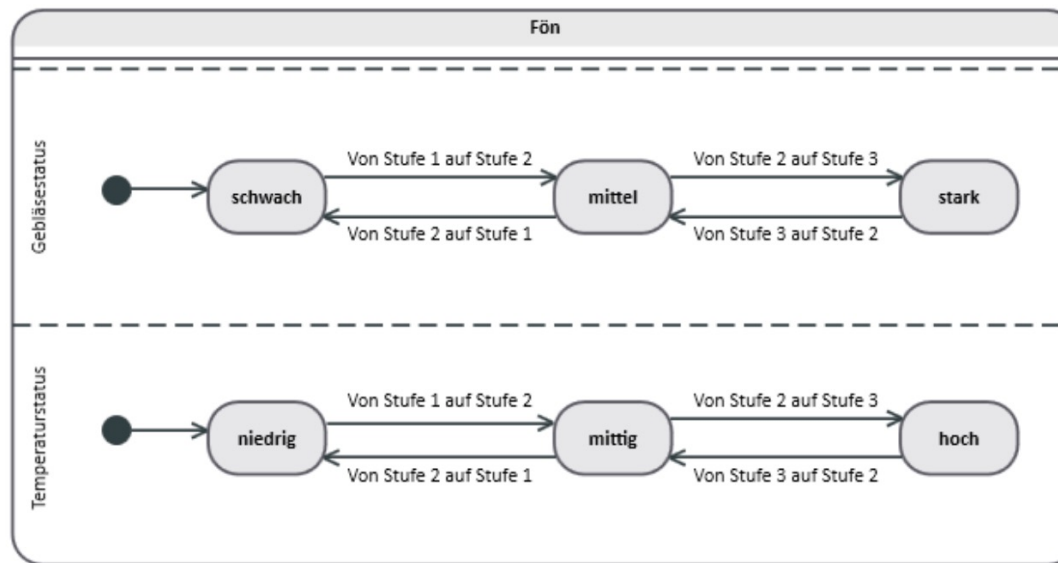


Modellierung von Anforderungen

Zustandsdiagramm: Beispiel



Nebenläufigkeit von Zustandsautomaten, d.h. der Superzustand kann in zwei parallel und unabhängig voneinander ablaufende Automaten A und B zerlegt werden.



Basiert auf einem Beispiel aus dem Buch von Tim Weilkiens:
„Systems Engineering mit SysML/UML“, dpunkt.verlag

Modellierung von Anforderungen

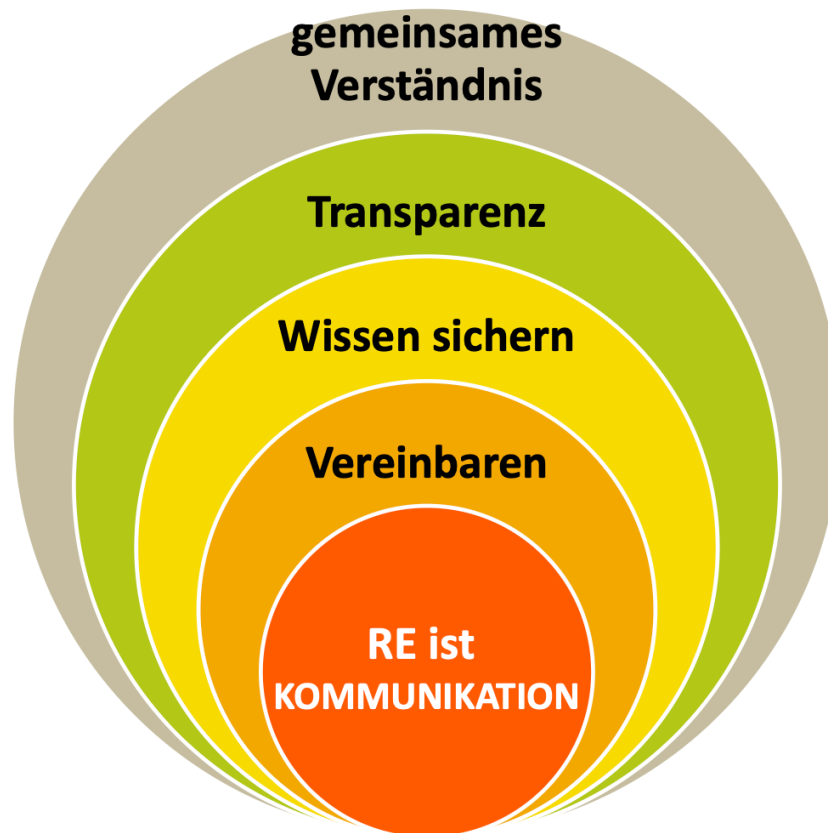
Gruppenaufgabe: Zustandsdiagramm

- Erstellen Sie ein Zustandsdiagramm, welches die verschiedenen Zustände des smarten Mähdreschers (oder eine seiner Subkomponenten) beschreibt.
- Zeitansatz: 10 Minuten

Kommunikation im RE

Kommunikation im RE

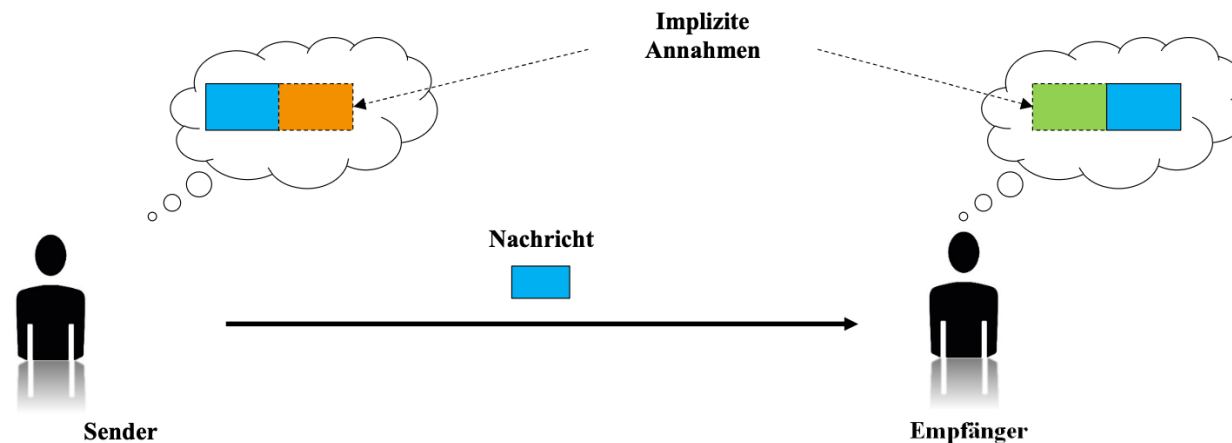
Die Kommunikations-Aspekte des RE



Anforderungen dienen als Grundlage für verschiedene Arten der Kommunikation, über den ganzen Lebenszyklus des Produkts!

Kommunikation im RE

Kommunikation: Implizite Annahmen



Sender und Empfänger ergänzen zu der tatsächlich gesendeten Nachricht jeweils ihre eigenen, impliziten Annahmen.

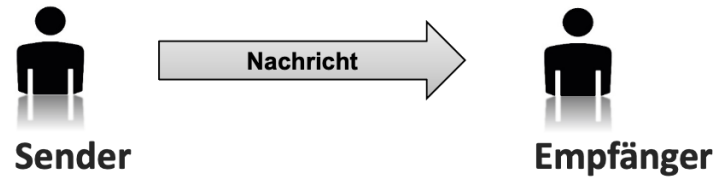
Die Folge: Beide haben unterschiedliche Bilder im Kopf, meinen aber über das Gleiche zu sprechen.

**Ein „Aussprechen“ dieser Annahmen vermeidet
Missverständnisse.**

Kommunikation im RE

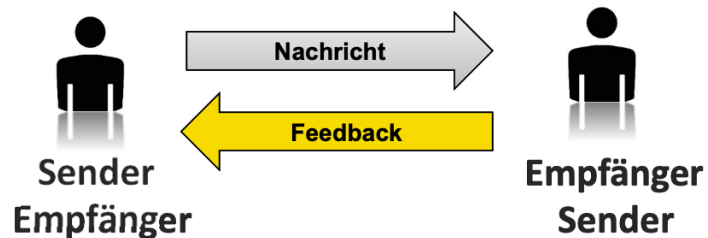
Kommunikationsmodelle

1. Einfaches Kommunikationsmodell



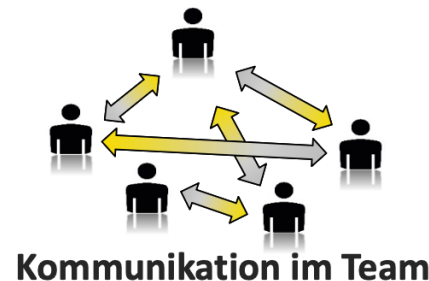
Vermeide das
„Über-den-
Zaun-werfen“!

2. Erweitertes Kommunikationsmodell



Gute verlässliche
Kommunikation
erfordert häufiges
Feedback!

3. Fortgeschrittenes Kommunikationsmodell



In kleinen Teams
findet dauerndes,
schnelles und
selbstorganisierte
Feedback statt!

Prüfen und abstimmen von Anforderungen

Prüfen von Anforderungen

Qualitätsaspekte

- Um eine für die nachfolgenden Entwicklungsschritte ausreichende Qualität der Anforderungen sicherzustellen,
- werden Anforderungen anhand vorher festgelegter Qualitätskriterien geprüft
- wird entschieden, ob die Qualität für eine Freigabe der Anforderungen ausreicht

➤ **Ziel der Prüfung:** Fehler in Anforderungen finden und beheben

Prüfen von Anforderungen

Kernziele

- Vollständigkeit sicherstellen
- Konsistenz zwischen Anforderungen prüfen
- Umsetzbarkeit bewerten
- Mehrdeutigkeiten eliminieren
- Nachvollziehbarkeit gewährleisten

Prüfen von Anforderungen

Kernziele

Kriterium	Beschreibung
Eindeutigkeit	Nur eine Interpretation möglich
Vollständigkeit	Alle notwendigen Informationen enthalten
Konsistenz	Keine Widersprüche zu anderen Anforderungen
Umsetzbarkeit	Technisch und wirtschaftlich realisierbar
Testbarkeit	Überprüfbar durch Tests oder Inspektionen
Nachverfolgbarkeit	Zuordnung zu Stakeholdern und Zielen möglich
Priorisierbarkeit	Dringlichkeit und Wichtigkeit bestimmbar

Prüfen von Anforderungen

Methoden zur Prüfung

- **Reviews und Inspektionen**

- Formale Inspektionen: Strukturiert mit Checklisten, Rollen (Moderator, Autor, Leser, Recorder)
- Walkthroughs: Autor führt durch die Anforderungen, Feedback wird gesammelt
- Peer Reviews: Kollegiale Prüfung ohne formellen Rahmen
- Gruppenreviews: Eine Gruppe prüft Anforderungen

- **Prototyping**

- Frühe Visualisierung zur Validierung mit Stakeholdern
- Besonders geeignet für funktionale Anforderungen und UI/UX

- **Modellbasierte Prüfung**

- Verwendung von Modellen (z.B. UML, SysML) zur formalen Analyse
- Automatische Konsistenzprüfungen möglich

- **Checklisten-basierte Prüfung**

- Standardisierte Fragen zur systematischen Durchsicht
- Beispiel: "Ist die Anforderung messbar definiert?"

Prüfen von Anforderungen

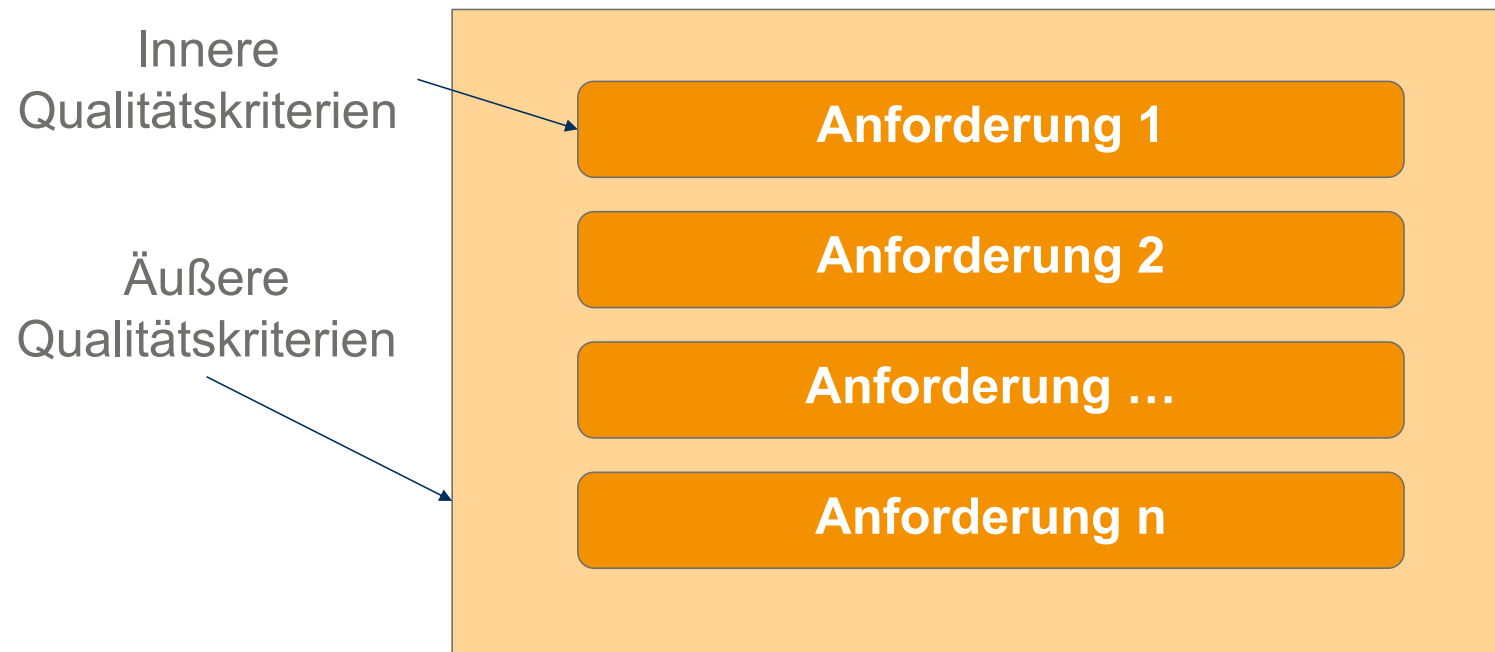
was soll die prüfung ergeben? was ist das ziel davon?

Grundsätzliche Vorgehensweise

- Kombinieren Sie **verschiedene Prüftechniken!**
- Führen Sie verschiedene Prüftechniken zu **verschiedenen Zeitpunkten** durch!
- Definieren Sie jeweils **passende Prüfkriterien!**

Prüfen von Anforderungen

Recap: Qualitätskriterien von Anforderungen



Wir brauchen Qualitätskriterien sowohl für die einzelnen Anforderungen als auch für das gesamte Anforderungsdokument.

Prüfen von Anforderungen

Recap: Qualitätskriterien

Innere Qualitätskriterien:

- vollständig
- widerspruchsfrei
- eindeutig
- realisierbar
- verständlich
- nachweisbar
- identifizierbar
- atomar
- redundanzfrei
- korrekt abgeleitet
- auf der korrekten Abstraktionsebene
- nachverfolgbar zur Quelle

Äußere Qualitätskriterien:

bezogen aufs Gesamtsystem

- Äußere Vollständigkeit
- Äußere Widerspruchsfreiheit
- Notwendigkeit
- Äußere Redundanzfreiheit

Prüfen von Anforderungen

Recap: Maßnahmen, um gute Anforderungen zu schreiben

M1 – Weak-Words vermeiden (aber, ziemlich,...)

M2 – Eigenschaftswörter hinterfragen und quantifizieren

M3 – Universalquantoren (alle, immer, nie,...) vermeiden bzw. konkretisieren

M4 – Nominalisierungen (Abläufe, Handlungen, allgemeine Begriffe...) mit einem Verb beschreiben oder präzisieren

M5 – W-Fragen stellen, um zu präzisieren

M6 – Bedingungen eindeutig und vollständig beschreiben

M7 – Im Aktiv formulieren

M8 – Kontextinformation getrennt von der Anforderung verwalten

M9 – Satzaufbau bewusst wählen – Satzschablone: [Bedingung +] das “System“ muss...+ Objekt/Verb/Beschreibung

M10 – Atomar formulieren

M11 – Möglichst das Wort „muss“ (engl. „shall“) verwenden

M12 – Anforderungen kennzeichnen und identifizierbar machen

M13 – Bezugspunkte bei Steigerungen und Vergleichen spezifizieren

Prüfen von Anforderungen

Grundlagen

- Die beim Prüfen von Anforderungen betrachteten Aspekte lassen sich grob in drei Qualitätsaspekte gruppieren:
- **Inhalt**
Innere Qualitätskriterien
- **Dokumentation**
 - Äußere Qualitätskriterien
 - Konformität zu Dokumentationsformat & Dokumentenstruktur, Dokumentationsregeln
- **Abgestimmtheit**
 - Abstimmung mit relevanten Stakeholdern
 - Abstimmung nach Änderungen
 - Konflikte aufgelöst

Bevor Anforderungen für die Verwendung in nachfolgenden Entwicklungsaktivitäten freigegeben werden, sollten diese drei Aspekte überprüft worden sein.

Abstimmen von Anforderungen

Abstimmen von Anforderungen

Notwendigkeit der Abstimmung von Anforderungen

Abstimmen und Konsolidieren von Anforderungen

- Sich einander widersprechende Anforderungen von verschiedenen Stakeholdern führen zu Konflikten.
- Werden sie nicht aktiv aufgelöst, führt dies zu Risiken im weiteren Projektverlauf (z.B. System wird von Stakeholdern nicht akzeptiert)

➤ **Ziel der Abstimmung:** Gemeinsames Verständnis und Einigung über die geforderten Eigenschaften des zu entwickelnden Systems unter allen relevanten Stakeholdern herbeiführen (Konsolidieren)

Abstimmen von Anforderungen

Notwendigkeit der Abstimmung von Anforderungen

Einige Gründe für die Notwendigkeit der Abstimmung:

- Vermeidung von Missverständnissen
- Konsistenz und Vollständigkeit der Anforderungen
- Einhaltung von Standards und Compliance
- Vermeidung von „Scope Creep“
- Steigerung der Akzeptanz und Zufriedenheit
- Risikominimierung
- Verbesserte Kommunikation und Transparenz
- Vermeidung von Konflikten
- Ausrichtung auf Unternehmensziele
- Nachhaltige Produktqualität
- Kundenzufriedenheit und Markterfolg

Addition von neuen Anforderung, Änderung der grundlegenden Ausrichtung, welches alles komplette über den Haufen wirft (z.b. Website > App)

Abstimmen von Anforderungen

Stakeholderperspektiven

Stakeholder	Perspektive	Typische Anforderungen	Risiko bei fehlender Abstimmung
Kunden	„Was brauche ich, um meine Ziele zu erreichen?“	Funktionalität, Benutzerfreundlichkeit, Performance.	Unzufriedenheit, Ablehnung des Produkts.
Product Owner	„Welche Anforderungen liefern den größten Mehrwert?“	Priorisierung, ROI, Marktfitt.	Falsche Priorisierung, verschwendete Ressourcen.
Entwicklungsteam	„Wie können wir die Anforderungen technisch umsetzen?“	Machbarkeit, technische Einschränkungen, Architektur.	Technische Schulden, Verzögerungen.
Management	„Wie trägt das Projekt zu den Unternehmenszielen bei?“	Budget, Zeitplan, Compliance.	Budgetüberschreitungen, strategische Fehlausrichtung.
Recht/Compliance	„Erfüllt das Produkt alle gesetzlichen und normativen Vorgaben?“	Datenschutz (GDPR), Sicherheitsstandards (ISO 26262), Branchenvorgaben.	Rechtliche Konsequenzen, fehlende Zulassungen.
Betrieb/IT	„Wie kann das Produkt in die bestehende Infrastruktur integriert werden?“	Skalierbarkeit, Wartbarkeit, Integration.	Inkompatibilitäten, hohe Wartungskosten.
Marketing/Vertrieb	„Wie kann das Produkt vermarktet werden?“	Unique Selling Points (USPs), Kundensegmentierung.	Schlechte Marktpositionierung, geringe Verkaufszahlen.

Abstimmen von Anforderungen

Methoden zur Abstimmung von Anforderungen

Methode	Beschreibung	Vorteile	Nachteile	Einsatzbereich
Workshops (z. B. JAD)	Gemeinsame Erarbeitung von Anforderungen in moderierten Sitzungen .	Kollaborativ, direktes Feedback.	Zeitaufwendig, erfordert Moderation.	Alle Projektarten.
Interviews	Einzelgespräche mit Stakeholdern zur Anforderungsermittlung.	Detaillierte Informationen, flexibel.	Zeitaufwendig, subjektiv.	Kleine bis mittlere Projekte.
Fragebögen	Schriftliche Befragung von Stakeholdern.	Skalierbar, anonym.	Geringe Rücklaufquote, oberflächlich.	Große Stakeholder-Gruppen.
Reviews	Formale Prüfung von Anforderungen (z. B. nach IEEE 830).	Systematisch, qualitätssichernd.	Aufwendig, erfordert Expertise.	Klassische Projekte.
Delphi-Methode	Anonyme Expertenbefragung in mehreren Runden zur Konsensfindung.	Reduziert Subjektivität, systematisch.	Zeitaufwendig.	Komplexe Entscheidungen.

Abstimmen von Anforderungen

Herausforderung bei der Abstimmung

Herausforderung	Ursache	Lösung
Unterschiedliche Fachsprachen	Stakeholder verwenden unterschiedliche Begriffe .	Glossar erstellen, Workshops durchführen.
Konfliktäre Interessen	Stakeholder haben unterschiedliche Prioritäten .	Priorisierungsmethoden (z. B. MoSCoW), Kompromisse finden.
Fehlende Zeit für Abstimmung	Stakeholder sind nicht verfügbar .	Effiziente Methoden (z. B. Fragebögen, asynchrone Reviews).
Subjektivität	Anforderungen werden emotional bewertet .	Objektive Methoden (z. B. AHP, Value Points).
Komplexität bei vielen Stakeholdern	Zu viele Beteiligte und Anforderungen .	Modularisierung, Stakeholder-Gruppen bilden.
Fehlende Dokumentation	Anforderungen werden nicht festgehalten .	Tools (z. B. Confluence, Jira) nutzen, Vorlagen verwenden.
Widerstand gegen Änderungen	Stakeholder akzeptieren keine Kompromisse .	Change Management, Transparente Kommunikation .

Abstimmen von Anforderungen

Konfliktmanagement

Die Abstimmung von Anforderungen erfordert ein systematisches Management von Konflikten. Das Konfliktmanagement lässt sich in folgende vier Aufgaben zerlegen:

- Konfliktidentifikation
- Konfliktanalyse
- Konfliktauflösung
- Dokumentation der Konfliktauflösung

Abstimmen von Anforderungen

Arten von Anforderungen

Um Konflikte gezielt auflösen zu können, ist es wichtig, sich der Art des Konflikts bewusst zu werden.

Mögliche Arten von Konflikten:

- Sachkonflikt
- Interessenskonflikt
- Wertekonflikt
- Beziehungskonflikt
- Strukturkonflikt

➤ In der Praxis besitzen die meisten Konflikte Anteile aus mehreren Konfliktarten.

Abstimmen von Anforderungen

Eskalationsstufen von Konflikten

Ebene	Beschreibung	Ausprägung
Ebene 1 : Win-Win	Konflikt soll noch im Einverständnis gelöst werden	1. Verhärtung 2. Debatte & Polemik 3. Taten statt Worte
Ebene 2: Win-Lose	Konflikt rückt in den Hintergrund; Durchsetzen eigener Standpunkt wird wichtiger.	4. Image & Koalition 5. Gesichtsverlust 6. Drohgebärden
Ebene 3: Lose-Lose	Ursprüngliches Konfliktthema (fast) vergessen. Es geht nur darum, dem Gegenüber zu schaden.	7. Begrenzte Vernichtungsschläge 8. Zersplitterung 9. Gemeinsame in den Abgrund

Abstimmen von Anforderungen

Konfliktlösung

Eine vollständige Auflösung eines Konfliktes ist nur möglich, wenn alle durch den Konflikt betroffenen Stakeholder in die Konfliktauflösung involviert werden. Es kann Sinn machen, dass eine hierarchische Eskalation den Konflikt für das Projekt/Initiative löst. Der Konflikt schwelt unter Umständen weiter, was im weiteren Fortschritt abträglich sein kann.

- Ein Konflikt kann mithilfe verschiedener Techniken gelöst werden
 - **Einigung** (auf eine der Alternativen)
 - **Kompromiss** (zwischen den Alternativen)
 - **Abstimmung** (Mehrheitsentscheidung)
 - **Variantenbildung** (jede Alternative wird in einer Systemvariante umgesetzt)
 - **Ober-sticht-Unter**
 - **Entscheidungsmatrix**
 - **Daumen-Veto Methode**

Abstimmen von Anforderungen

Dokumentation von Konflikten (-lösungen)

- Ein Konflikt und seine Auflösung sollten nachvollziehbar dokumentiert sein: Klären des Warum
- Hierzu sollten folgende Punkte festgehalten werden:
 - insbesondere die Konfliktursache,
 - die beteiligten Stakeholder,
 - die Meinungen der einzelnen Stakeholder,
 - die Art der Konfliktauflösung,
 - mögliche Alternativen,
 - die Entscheidungen und
 - die Gründe für die Entscheidungen.

Verwaltung von Anforderungen

Verwaltung von Anforderungen

Aufgaben des RE

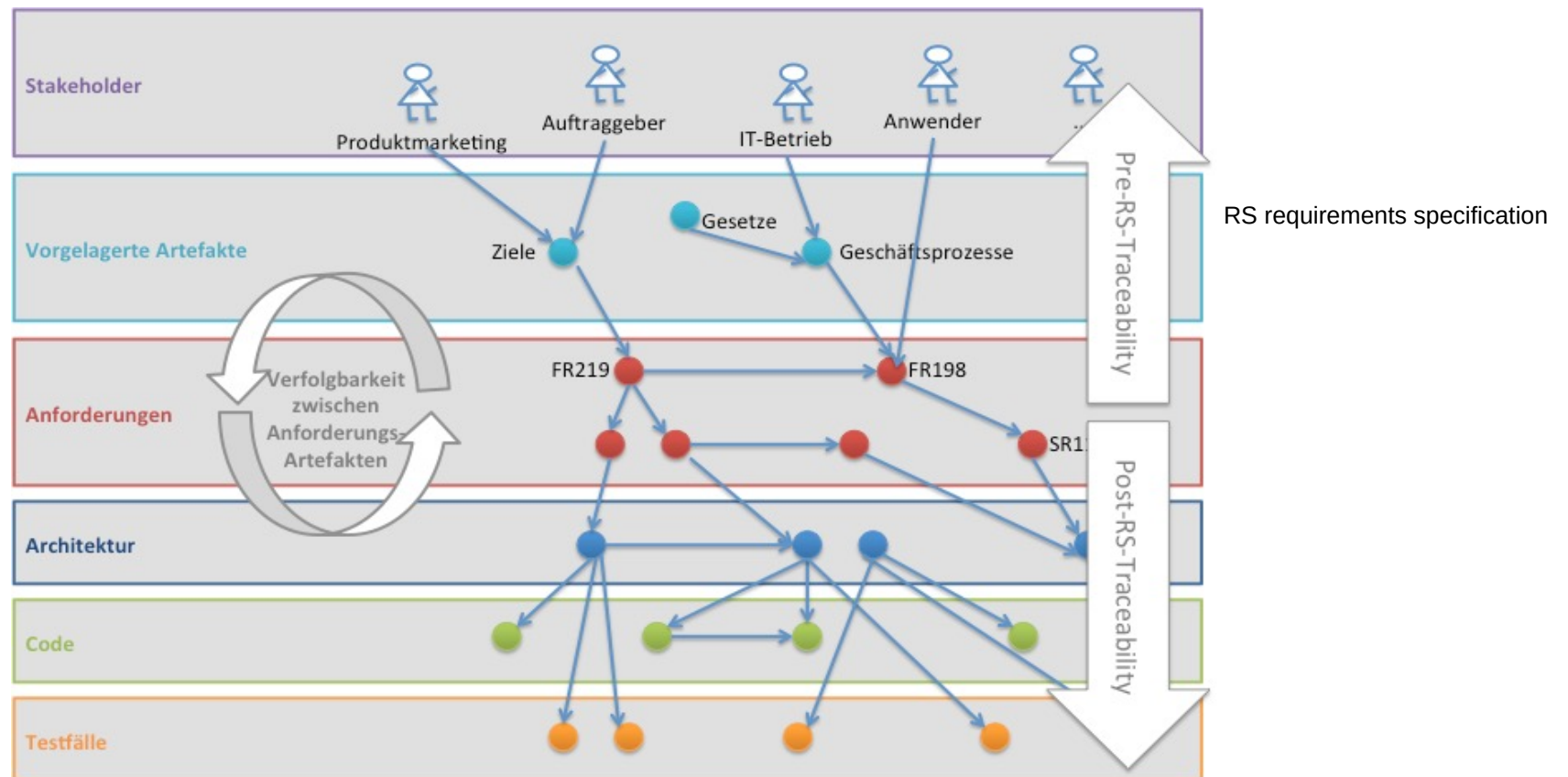
Das RM liefert unter anderem Antworten auf die nachfolgenden Fragestellungen:

- Welche Anforderungsarten werden unterschieden? (Anforderungslandschaft)
- Auf welchen Abstraktionsgraden werden Anforderungen dokumentiert? (Anforderungslandschaft)
- Welche Anforderungen sind bereits abgenommen? (Attributierung)
- Welche Anforderungen stammen aus welcher Quelle? (Attributierung)
- Welche Anforderungen sind dringend und wichtig und somit Kandidaten für das nächste Release? (Bewertung und Priorisierung)
- Welche Anforderungen erzeugen hohe Kosten bei geringem Nutzen? (Bewertung und Priorisierung)
- Welche Anforderungen gehören zu einer bestimmten Basislinie der Software? (Versionsverwaltung)

- Welche Version der Anforderung wurde im System umgesetzt? (Versionierung)
- Wer hat die Anforderung zuletzt verändert und warum? (Versionierung)
- Welche technische Komponente gehört zu welcher Anforderung? (Verfolgbarkeit)
- Welche Testfälle gehören zu welcher Anforderung? (Verfolgbarkeit)
- Welche Anforderung ist Teil des ausgelieferten Systems / Produkts? (Verfolgbarkeit)
- Worin unterscheiden sich die beiden Varianten des Produkts? (Variantenmanagement)
- Welcher Anteil der Anforderungen wurde bereits umgesetzt und getestet? (Berichtswesen)
- Wie lange braucht ein Change Request im Durchschnitt bis zu seiner Umsetzung? (Berichtswesen)
- Hat sich der RE-Prozess durch eine bestimmte Maßnahme verbessert? (Berichtswesen)

Verwaltung von Anforderungen

Verfolgbarkeit - Beispiel

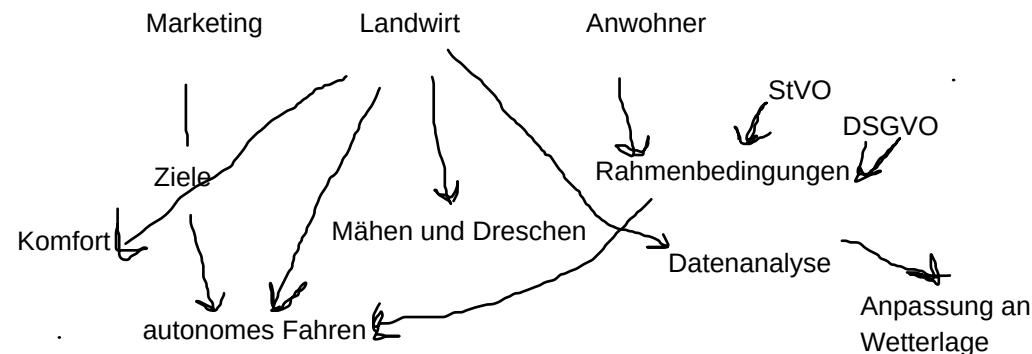


Verwaltung von Anforderungen

Gruppenaufgabe

- Sie haben erste Anforderungen für unseren smarten Mähdrescher erstellt. Dieser soll nun entwickelt und gebaut werden.
- Definieren Sie in Ihrem Informationsmodell sinnvolle Traceability zwischen den Dokumenten. Zeichnen sie dazu Traceability-Pfeile zwischen den Artefakten und beschriften Sie sie mit ihrer Bedeutung („wird abgeleitet von“, „weist nach“, „wird nachgewiesen von“,).
- Tipp:
 - Beschränken Sie sich auf wenige Link-Typen (auch in der realen Praxis!)
 - Überlegen Sie sich zu jeder Traceability, was Sie damit bezwecken wollen (auch in der realen Praxis!)

Zeitansatz: 15 Minuten



Verwaltung von Anforderungen

Verfolgbarkeit von Anforderungen

- Verfolgbarkeit (Traceability) ist die Fähigkeit, Anforderungen rückverfolgbar über den gesamten Lebenszyklus des Systems zu dokumentieren – von der Erhebung über die Implementierung bis zur Abnahme.
 - Vorwärtsverfolgbarkeit: Von der Anforderung zur Implementierung (z. B. „Anforderung #42 → Code-Modul X“).
 - Rückwärtsverfolgbarkeit: Von der Implementierung zur Anforderung (z. B. „Fehler in Modul X → Welche Anforderung ist betroffen?“).
- **Nutzen von Verfolgbarkeit**
 - Transparenz: Nachvollziehbarkeit für Stakeholder, Entwicklern und Auditoren.
 - Änderungsmanagement: Auswirkungen von Änderungen auf andere Anforderungen oder Komponenten erkennen.
 - Compliance: Nachweis der Erfüllung von Standards (z. B. ISO 26262 für Automotive, IEC 62304 für Medizintechnik).
 - Fehleranalyse: Schnelle Identifikation betroffener Anforderungen bei Fehlern oder Änderungen.

Verwaltung von Anforderungen

Diskussion

„Welche Probleme können entstehen, wenn Anforderungen nicht verfolgbar sind bzw. nicht strukturiert verfolgt werden?“

Verwaltung von Anforderungen

Repräsentation von Verfolgbarkeit

Textuelle Referenzen und Hyperlinks

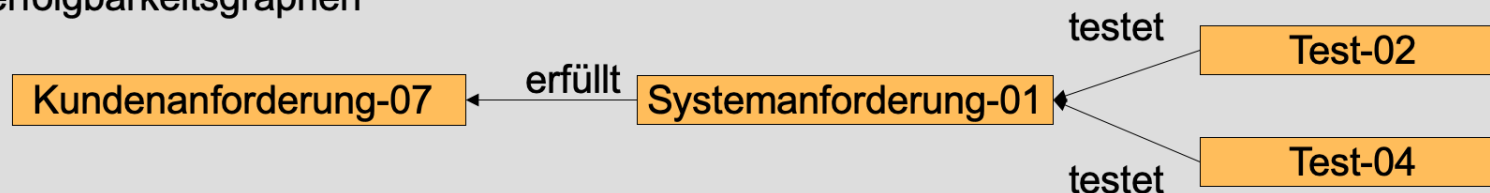
Wenn die Ampel rot ist muss das System das Fahrzeug innerhalb von 10 Sekunden zum stehen bringen (siehe auch Test T_0238)

Quelle: [S-784](#)

Verfolgbarkeitsmatrizen

	01 Test-	02 Test-	03 Test-	04 Test-
Anf-01		x		x
Anf-02			x	

Verfolgbarkeitsgraphen



Verwaltung von Anforderungen

Traceability-Matrix

Anforderungs-ID	Beschreibung	Design-Dokument	Code-Modul	Testfall	Status
REQ-001	Benutzer kann sich anmelden	DESIGN-05	AuthService.java	TEST-12	Implementiert
REQ-002	Passwort muss verschlüsselt werden	DESIGN-07	PasswordUtils.java	TEST-15	In Review

Verwaltung von Anforderungen

Prüfung der Verfolgbarkeit in verschiedenen Projektphasen



Frühe Phase (Elicitation): Validierung mit Stakeholdern (z. B. durch Interviews).



Analysephase: Konsistenzprüfung zwischen Anforderungen (z. B. mit Traceability-Matrizen).



Implementierungsphase: Akzeptanztests und Code-Reviews.



Abnahmephase: Akzeptanztests mit Stakeholdern.

Verwaltung von Anforderungen

Herausforderungen in der Verwaltung von Anforderungen

Herausforderung	Ursache	Lösung
Hoher Aufwand für Pflege	Manuelle Aktualisierung der Matrix	Automatisierte Tools (z. B. Jama Connect)
Inkonsistente IDs	Keine einheitlichen Konventionen	Styleguides und Vorlagen
Fehlende Akzeptanz im Team	Unklarer Nutzen	Schulungen und Pilotprojekte
Komplexität bei großen Projekten	Viele Abhängigkeiten	Modulare Traceability (z. B. pro Subsystem)

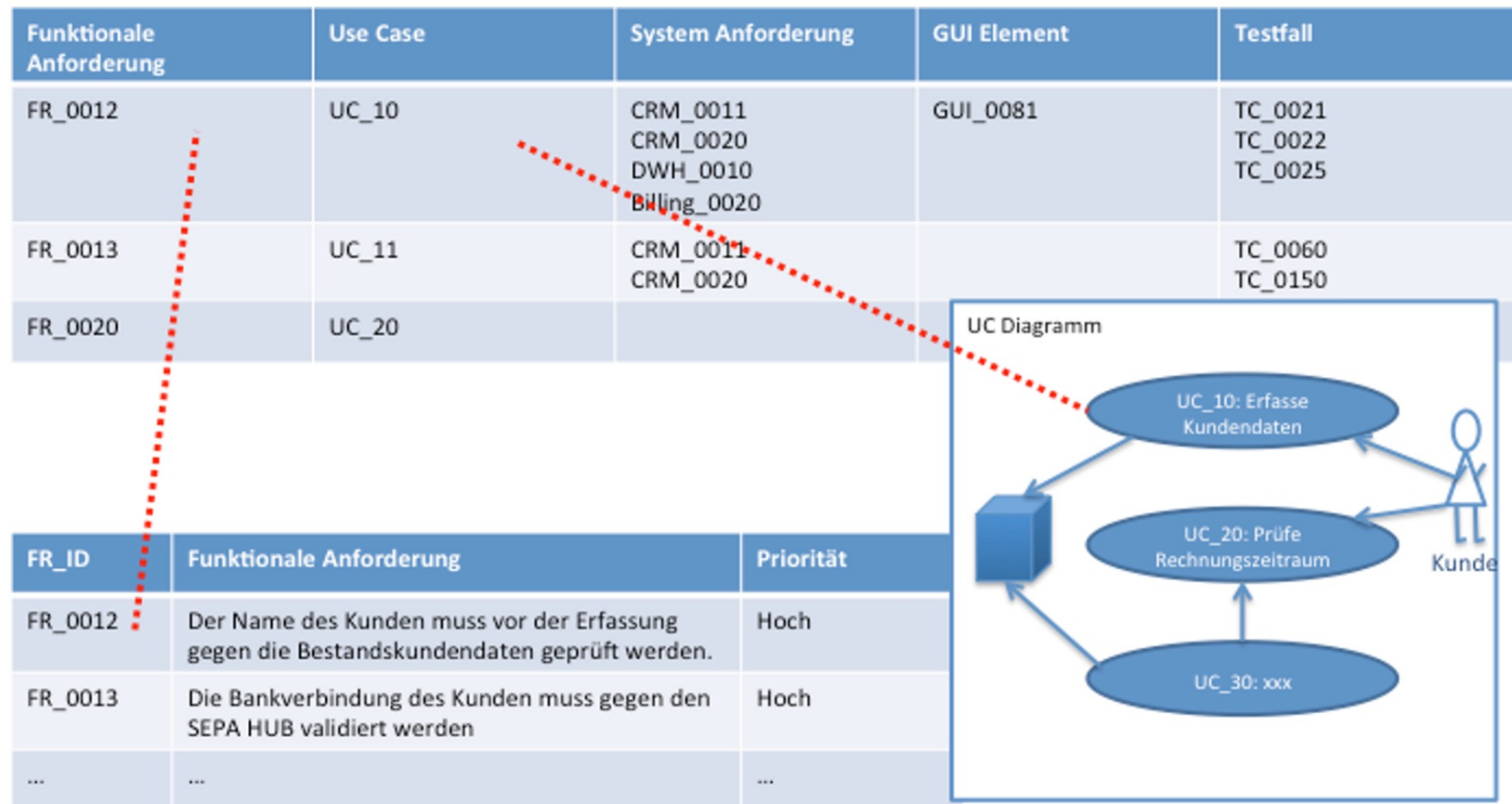
Verwaltung von Anforderungen

Software-Werkzeuge

Tool	Einsatzbereich	Vorteile	Nachteile
Excel	Kleine Projekte	Einfach, keine Kosten	Manuell, fehleranfällig
Confluence	Dokumentation	Integration mit Jira, kollaborativ	Begrenzte Automatisierung
Jira	Agile Projekte	Verknüpfung mit User Stories	Komplex für große Traceability
IBM DOORS	Klassisches RE, große Projekte	Mächtige Traceability-Features	Teuer, steile Lernkurve
Jama Connect	Medizinprodukte, Luftfahrt	Echtzeit-Kollaboration, Audits	Hohe Kosten

Verwaltung von Anforderungen

Verfolgbarkeit zwischen textuellen und modellbasierten Artefakten



Verwaltung von Anforderungen

Attribute

In der Literatur finden sich verschiedene Sätze an Attributen. Das IREB definiert folgende Attribute als wichtig:

- Identifikator
- Beschriftung
- Autor
- Begründung
- Kritikalität
- Name
- Version
- Quelle
- Stabilität
- Priorität

Verwaltung von Anforderungen

Attributierung von Anforderungen

In der Literatur finden sich verschiedene Sätze an Attributen. Das IREB definiert ***folgende Attribute als wichtig:***

- Identifikator
- Beschriftung
- Autor
- Begründung
- Kritikalität
- Name
- Version
- Quelle
- Stabilität
- Priorität

Ergänzende Attribute:

- Verantwortlich/Verantwortlicher
- Anforderungstyp
- Status Inhalt
- Status Einigung
- Status Überprüfung
- Release
- (Quer-)Bezüge
- Aufwand
- Jur. Verbindlichkeit
- Kommentar

Verwaltung von Anforderungen

Nutzen von Attributen

Attribute unterstützen bei einer Vielzahl an :



Verwaltung von Anforderungen

Attributierungsschema

Anforderungen und Zusatzinformationen zu Anforderungen müssen über den gesamten Lebenszyklus eines Produktes effizient verwaltet werden können.

→ eine strukturierte Erfassung ist essentiell

Mithilfe eines Attributierungsschemas wird definiert, welche Arten von Informationen zu einer Klasse von Anforderungen erfasst werden sollen und welche Wertebereiche diese Informationen jeweils besitzen dürfen

z.B. Attribut „Priorität“ mit dem Wertebereich „sehr hoch, hoch, niedrig, sehr niedrig, n/a, tbd“

Verwaltung von Anforderungen

Beispiel für Attributierungsschema

Attribut	Typ	Gültige Werte	Default-Wert	Werte mehrfach auswählbar	Pflichtattribut/ optionales Attribut (beim Anlegen der Anforderung)
Beschreibung	Freitext	-		-	ja
Status	Enumeration	• in Bearbeitung • in Prüfung • freigegeben	In Bearbeitung	nein	ja
Risiko	Enumeration	• tbd • niedrig • mittel • hoch	tbd	nein	nein

Verwaltung von Anforderungen

Best Practices

Konsistente Namenskonventionen

Beispiel: PROJ-REQ-001 (Projektkürzel + Typ + Nummer).
Vermeidung von Mehrdeutigkeiten (z. B. REQ1 vs. REQ-001).

Pflichtfelder

Definition, welche Attribute immer gepflegt werden müssen (z. B. ID, Beschreibung, Priorität, Status).

Automatisierung

Tools wie Jira oder IBM DOORS können Attribute automatisch verknüpfen (z. B. mit Code oder Testfällen).

Regelmäßige Reviews

Regelmäßige Überprüfung Attribute in Reviews auf Aktualität.

Schulungen für Stakeholder

Beteiligte (z. B. Entwickler, Tester) müssen die Bedeutung der Attribute verstehen.

Dokumentation

Bereitstellung eines Glossar oder Handbuch, das die Bedeutung jedes Attributs erklärt.

Verwaltung von Anforderungen

Gruppenaufgabe

Um Ihre Anforderungen vollständig zu dokumentieren, damit Sie diese im weiteren Projektverlauf auch sinnvoll nutzen können, ist es nun an der Zeit sich Gedanken darüber zu machen, wie die Dokumentation der Anforderungen erfolgen soll.

Erstellen Sie dazu ein **Attributierungsschema** und überlegen Sie sich für eine Komponente unseres smarten Mähdreschers sinnvolle Attribute.

Definieren Sie außerdem zu jedem Attribut die **gültigen Attributwerte** und **relevante Eigenschaften**.

Zeitansatz: 20 Minuten

Priorisierung von Anforderungen

Priorisierung von Anforderungen

Nutzen von Priorisierung

- Unterstützt die Entwicklungsplanung (Muster, Releases)
- Konflikte, vor allem später in der Entwicklung, können einfacher gelöst werden
- Die Konsequenzen der Nichterfüllung von Anforderungen können einfacher beurteilt werden
- Bildet Basis für die Entscheidung, welche Anforderungen getestet werden sollen und welche nicht

Priorisierung von Anforderungen

Priorisierungsmethoden - I

Technik	Beschreibung	Vorteile	Nachteile	Einsatzbereich	Beispiel
MoSCoW	Klassifizierung in Must have, Should have, Could have, Won't have .	Einfach, intuitiv, gut für Stakeholder-Kommunikation, klare Entscheidungsgrundlage.	Subjektiv, keine quantitative Bewertung.	Alle Projektarten, besonders in der Anforderungsanalyse und Lastenheft-Erstellung.	In einem Bankensystem: „Sicherheitszertifizierung“ = Must have, „Benachrichtigungen“ = Should have.
Kano-Modell	Unterscheidung in Grundanforderungen, Leistungsanforderungen, Begeisterungsanforderungen .	Berücksichtigt Kundenzufriedenheit, hilft bei der Identifikation von USP (Unique Selling Points).	Komplexer als MoSCoW, erfordert Marktkennntnis.	Produktentwicklung, Marketing, Kundenzufriedenheitsanalyse.	In einer E-Commerce-App: „Zahlungsabwicklung“ = Grundanforderung, „KI-Empfehlungen“ = Begeisterungsanforderung.
AHP (Analytic Hierarchy Process)	Paarweiser Vergleich von Anforderungen mit mathematischer Gewichtung	Objektiv, mathematisch fundiert, reduziert Subjektivität, gut für komplexe Entscheidungen.	Aufwendig, erfordert Expertise in der Anwendung.	Komplexe Projekte, strategische Entscheidungen, Portfolio-Management.	Vergleich von „Patientendatensicherheit“ vs. „Benutzerfreundlichkeit“.

Priorisierung von Anforderungen

Priorisierungsmethoden - II

Technik	Beschreibung	Vorteile	Nachteile	Einsatzbereich	Beispiel
Value Points (Nutzen-Aufwand-Analyse)	Bewertung nach Nutzen (1–10) und Aufwand (1–10), Priorisierung nach dem Verhältnis Nutzen/Aufwand.	Quantitativ, ROI-fokussiert, transparent.	Aufwand für Datenerhebung, subjektive Bewertung von Nutzen/Aufwand.	Wirtschaftliche Projekte, Ressourcenplanung, Budgetverteilung.	„Zahlungsabwicklung“ (Nutzen: 10, Aufwand: 5 → Verhältnis: 2.0) vs. „Dark Mode“ (Nutzen: 4, Aufwand: 2 → Verhältnis: 2.0).
Bubble Sort / Ranking	Manuelles Sortieren von Anforderungen nach Wichtigkeit in einer Liste.	Einfach, schnell, keine Tools erforderlich.	Subjektiv, keine Quantifizierung, schwer nachvollziehbar.	Kleine Projekte, schnelle Entscheidungen in Workshops.	1. Benutzerauthentifizierung, 2. Datenbankanbindung, 3. UI-Design.
100-Dollar-Methode	Stakeholder verteilen 100 Punkte auf Anforderungen nach Priorität.	Kollaborativ, transparent, fördert Konsensfindung.	Subjektiv, erfordert Moderation.	Workshops mit Stakeholdern, Konsensfindung.	„Login-Funktion“: 40 Punkte, „Datenexport“: 30 Punkte, „Dark Mode“: 10 Punkte.

Priorisierung von Anforderungen

Priorisierungsmethoden - III

Technik	Beschreibung	Vorteile	Nachteile	Einsatzbereich	Beispiel
Delphi-Methode	Expertenbefragung in mehreren Runden zur Konsensfindung (anonym, iterativ).	Reduziert Subjektivität, berücksichtigt Expertenmeinungen, systematisch.	Zeitaufwendig, erfordert Moderation.	Komplexe oder unsichere Projekte, Forschung & Entwicklung.	In einem Forschungsprojekt: Experten bewerten die Priorität von Algorithmen.
QFD (Quality Function Deployment)	Matrix zur Abbildung von Kundenanforderungen vs. technischen Lösungen , Priorisierung basierend auf Kundenbedürfnissen.	Systematisch, kundenfokussiert, hilft bei der Umsetzung von Kundenwünschen in technische Anforderungen.	Aufwendig, erfordert detaillierte Kundenanalysen.	Produktentwicklung (z. B. Automotive, Medizintechnik), Qualitätsmanagement.	In der Automobilindustrie: Kundenanforderung „Sicherheit“ wird mit technischen Lösungen wie „Airbags“ verknüpft.
Eisenhower-Matrix	Einteilung in dringend/wichtig (4 Quadranten).	Fokus auf Zeitmanagement, einfache Visualisierung.	Weniger detailliert für RE, keine quantitative Bewertung.	Zeitkritische Projekte, Entscheidungsfindung.	Dringend & wichtig: „Sicherheitsupdate“; Wichtig, aber nicht dringend: „UI-Optimierung“.

Priorisierung von Anforderungen

Gruppenaufgabe

1. Definieren Sie ein oder zwei Kriterien zur Priorisierung der zuvor erstellten Anforderungen
2. Führen Sie die Priorisierung durch

Zeitansatz: 15 Minuten

Anforderungen versionieren

Versionierung von Anforderungen

Versionierungstabelle - I

Version	Datum	Name	Durchgeführte Änderung
0.1	19.09.2014	Reber	Initiale Version
0.2	20.09.2014	Reber	Änderungen an Anforderung Req-0010, 0011, 0030, 0090
0.3	30.09.2014	Reber	Änderungen an der Priorität der Anforderungen
1.0	02.10.2014	Reber	Version zum Ersten Review erstellt
1.1	15.10.2015	Reber	Änderungen auf Basis der Review-Ergebnisse eingepflegt an Req-0030, 0034, 0035, 0089, 0090

Versionierung von Anforderungen

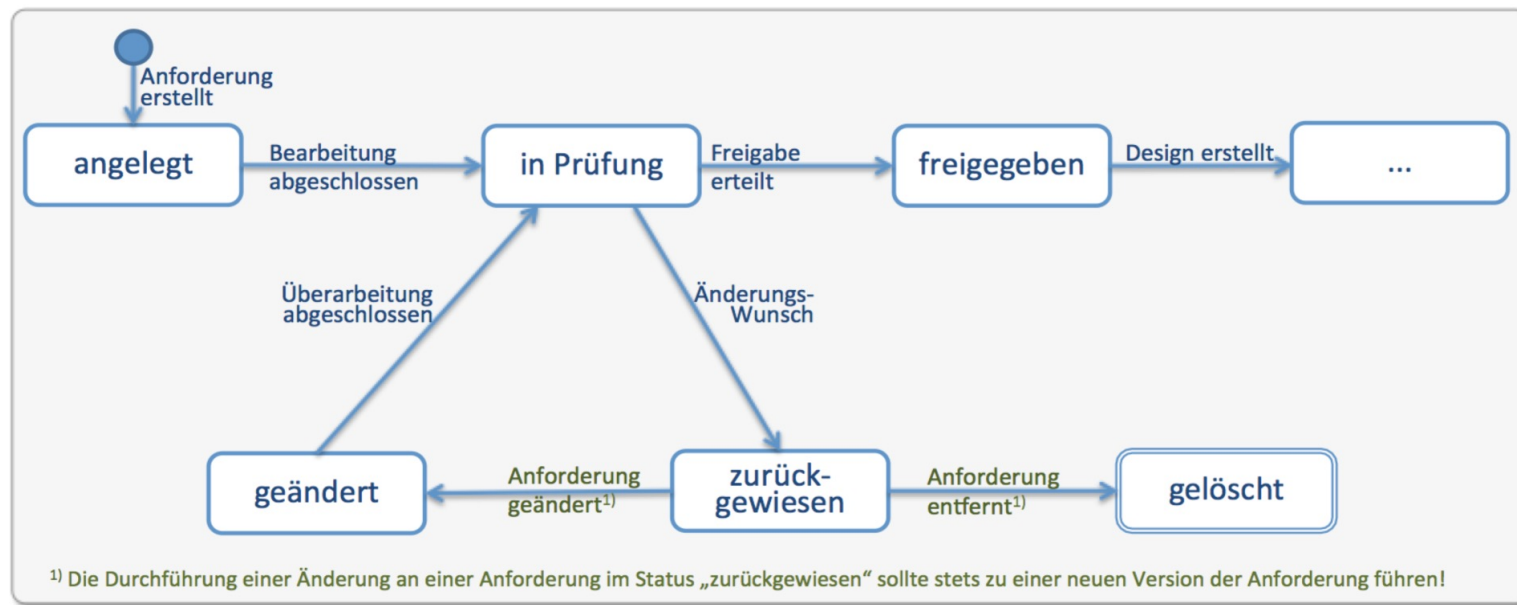
Versionierungstabelle - II

Req-ID	Datum	Version	Name	Grund der Änderung	Aktion	Anforderung
Req-30	19.09.2014	1	Reber		Angelegt	Das System soll a
Req-30	20.09.2014	2	Reber	Änderungen aufgrund neuer Information vom Fachbereich...	Geändert	Das System soll b
Req-30	15.10.2014	3	Reber	Änderung aufgrund Review von Max Müller	Geändert	Das System soll c
...						

Versionierung von Anforderungen

Zustände von Anforderungen

Im Verlauf des Lebenszyklus einer Anforderung oder Anforderungsspezifikation, weist diese unterschiedliche Zustände auf, die über entsprechende Zustandsübergänge erreicht werden.



Versionierung von Anforderungen

Freigabe von Untermengen von Anforderungen

In der Praxis müssen nicht alle Anforderungen am Anfang im Entwicklungsprozess freigegeben sein. Mehrere Stufen der Freigabe / des Fixierens oder Einfrierens von Anforderungen zu definieren, ist dafür hilfreich. Dies muss zusammen mit dem Geschäftsprozessen und dem Projektmanagement geschehen.

Beispiele für die Freigabe von Untermengen von Anforderungen (Meilensteine)

1. Begeisterungsfaktoren
2. Anforderungen, die sich vom Vorgänger-System unterscheiden
3. Anforderungen mit den höchsten Risiken für die Entwicklung und das Projekt
4. Anforderungen, die nur für den 1. Prototypen relevant sind
5. Kapitelweise Aufteilung von Anforderungen (Gesamtsystemanforderungen vor Detailsystemanforderungen)
6. Kategorieweise Aufteilung von Anforderungen (Dokumentations-Anforderungen erst später)

Änderungsmanagement / Change Management

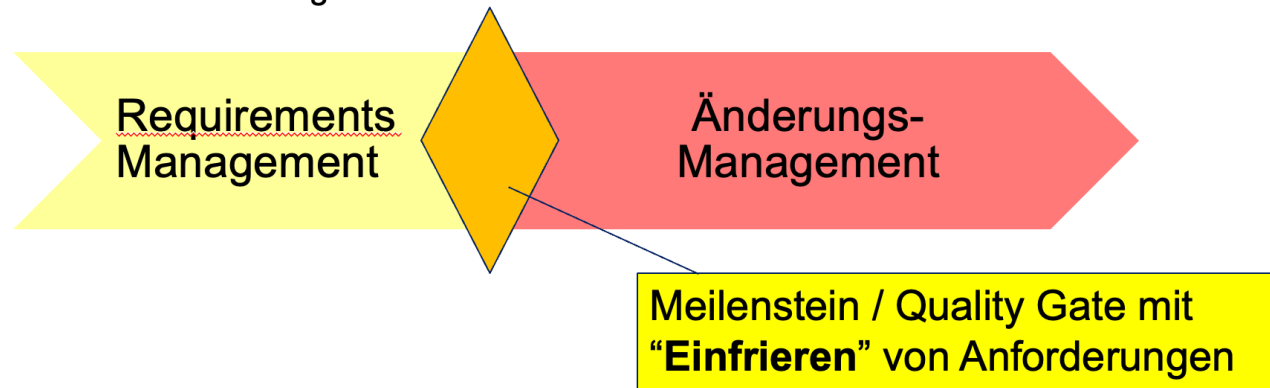
Versionierung von Anforderungen

Änderungsmanagement

Änderungsmanagement startet, sobald die **Anforderungen mit den Stakeholdern abgestimmt und freigegeben** sind. Damit werden sie unveränderbar und „eingefroren“, um unkontrollierte Veränderungen zu vermeiden.

Typischerweise ist dies in phasenorientierter Vorgehensweise mit einem **Meilenstein** oder einem **Quality Gate** verbunden.

Für **iterative oder agile Prozesse** sollte es ebenfalls definierte Zeitpunkte geben, ab dem relevante Anforderungen für die Iteration festgeschrieben sind.



Versionierung von Anforderungen

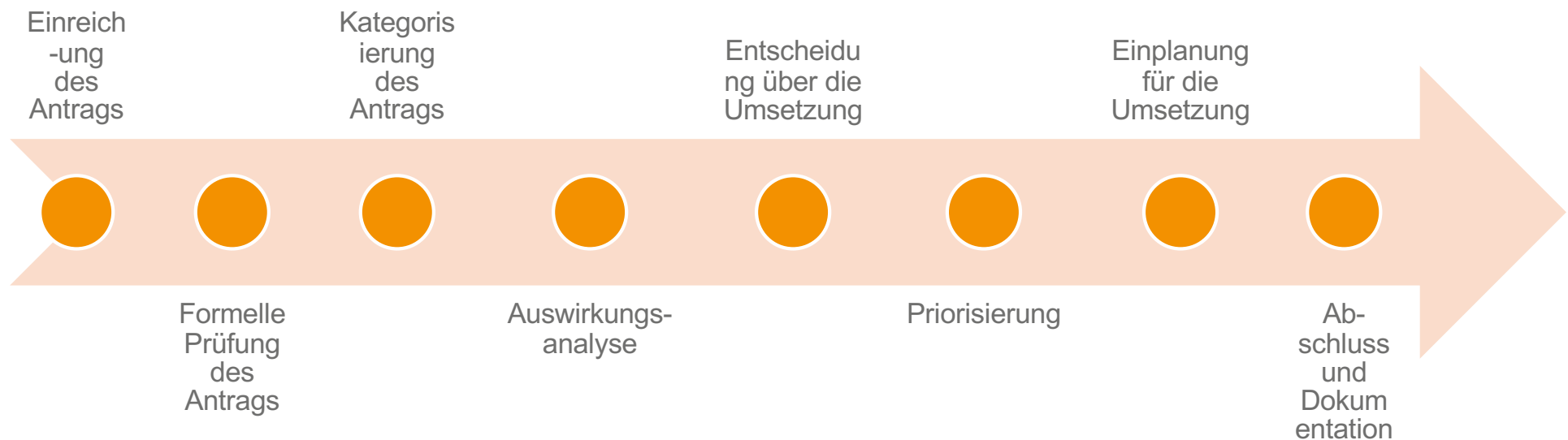
Änderungsmanagement / Change Management

Änderungsmanagement (Change Management) ist ein wichtiger Bestandteil des RE, um Kontrolle, Transparenz und Stabilität in Projekten zu gewährleisten

- Nachverfolgbarkeit (Traceability)
- Vermeidung von Chaos und Inkonsistenzen
- Risikominimierung
- Einhaltung von Compliance und Standards
- Transparenz und Kommunikation
- Schutz vor „Scope Creep“
- Ressourcenplanung

Versionierung von Anforderungen

Change Request-Prozess



Versionierung von Anforderungen

Change-Control-Board

- Das **Change-Control-Board (CCB)** ist verantwortlich für die Bearbeitung eingehender Änderungsanträge, insbesondere in größeren Initiativen/Projekten
- Die Aufgaben des **CCB** sind:
 - Klassifikation eingehender Änderungsanträge
 - Bestimmung des Aufwands einer Änderung
 - Beurteilung der Änderungsanträge hinsichtlich Implikationen, Aufwand & Nutzen
 - Definition neuer Anforderungen auf Basis eingehender Änderungsanträge
 - Entscheidung über Annahme oder Ablehnung eines Änderungsantrags
 - Priorisierung der angenommenen Änderungsanträge
 - Zuordnung der Änderungen zu Änderungsprojekten

Versionierung von Anforderungen

Change Request / Änderungsantrag

Inhalt	Beschreibung
Projektname	Bezeichnung des Projekts für welches die angefragte Änderung gilt.
Antragsnummer	Fortlaufende Nummer der Änderungsanträge innerhalb eines Projektes
Titel	Titel der gewünschten Änderung
Datum	Datum des Änderungsantrags
Antragsteller	Name des Antragstellers
Ursprung	Quelle bzw. Ursprung der Änderung (z.B. Marketing, Management, Kunde, Test)
Fachliche Verantwortung	Name oder Abteilung für die fachliche Verantwortung der Ursprungsfunktionalität
Änderungsart	Art der Änderungsanfrage (z.B. Defect, Innovation, Tuning)
Status	Aktueller Status der Änderungsanfrage (z.B. eingereicht, bewertet, abgelehnt, überprüft)
Priorität des Antragstellers	Priorität der Änderung aus der Sicht des Antragstellers
Priorität der Umsetzung	Priorität der Änderung aus der Sicht des Änderungsgremiums
Prüfer der Änderungsanfrage	Name der Person, welche die Durchführung (inkl. Auswirkung) der Änderung prüft
Aktualisierungsdatum	Datum der letzten Aktualisierung der Änderungsanfrage
Version	Versionsnummer des Änderungsdokuments
Release	Release-Zuordnung, zudem die Änderung umgesetzt werden soll
Änderungsaufwand in der Spezifikation	Prognostizierter Aufwand der Änderung
Änderungsaufwand in der Umsetzung	Prognostizierter Aufwand der Änderung
Beschreibung der Änderung	Allgemeine Beschreibung, oder: Was, wie und warum geändert/gelöscht/hinzugefügt werden soll
Kommentare	Kommentare zum Änderungsauftrag

Management von Varianten

Management von Varianten

Variantenmanagement

- Variantenmanagement ist der systematische Umgang mit unterschiedlichen Ausprägungen eines Produkts oder Systems, die sich in Funktionalität, Design oder Konfiguration unterscheiden, aber auf einer gemeinsamen Basis (z. B. einer Plattform oder einem Kernsystem) aufbauen.
- **Ziel:** Wiederverwendung von Anforderungen, Komponenten und Lösungen bei gleichzeitig flexibler Anpassung an spezifische Kunden- oder Marktbedürfnisse.
- Beispiele:
 - **Automobilindustrie:** Ein Basismodell (z. B. VW Golf) mit Varianten wie „Benzin“, „Diesel“, „Elektro“ oder „Sportversion“.
 - **Software:** Eine Standard-Software (z. B. SAP) mit Branchenlösungen (z. B. für Banken, Handel, Produktion).
 - **Medizintechnik:** Ein Grundgerät mit länderspezifischen Anpassungen (z. B. unterschiedliche Stromspannungen oder Sprachversionen).

Management von Varianten

Relevanz von Variantenmanagement

Aspekt	Nutzen	Beispiel
Kosteneinsparung	Reduziert Redundanzen durch Wiederverwendung von Anforderungen und Komponenten.	Gleiche Basis-Anforderungen für alle Varianten eines Autos (z. B. „Bremsystem“).
Flexibilität	Ermöglicht schnelle Anpassung an Kundenwünsche oder Marktveränderungen.	Neue Variante eines Smartphones mit zusätzlicher Kamera-Funktion.
Zeitersparnis	Beschleunigt die Entwicklung neuer Varianten durch Nutzung bestehender Artefakte.	Eine neue Software-Version für einen anderen Markt basierend auf einer bestehenden Variante.
Qualitätssicherung	Konsistenz zwischen Varianten durch zentrale Verwaltung von Anforderungen.	Alle Varianten eines Medizingeräts erfüllen die gleichen Sicherheitsanforderungen.
Compliance	Erleichtert die Einhaltung von Standards (z. B. ISO 26262, IEC 62304) für alle Varianten.	Alle Varianten eines Autos erfüllen die gleichen Crash-Test-Normen.
Kundenzufriedenheit	Ermöglicht maßgeschneiderte Lösungen für unterschiedliche Zielgruppen.	Eine Software mit spezifischen Modulen für kleine vs. große Unternehmen.

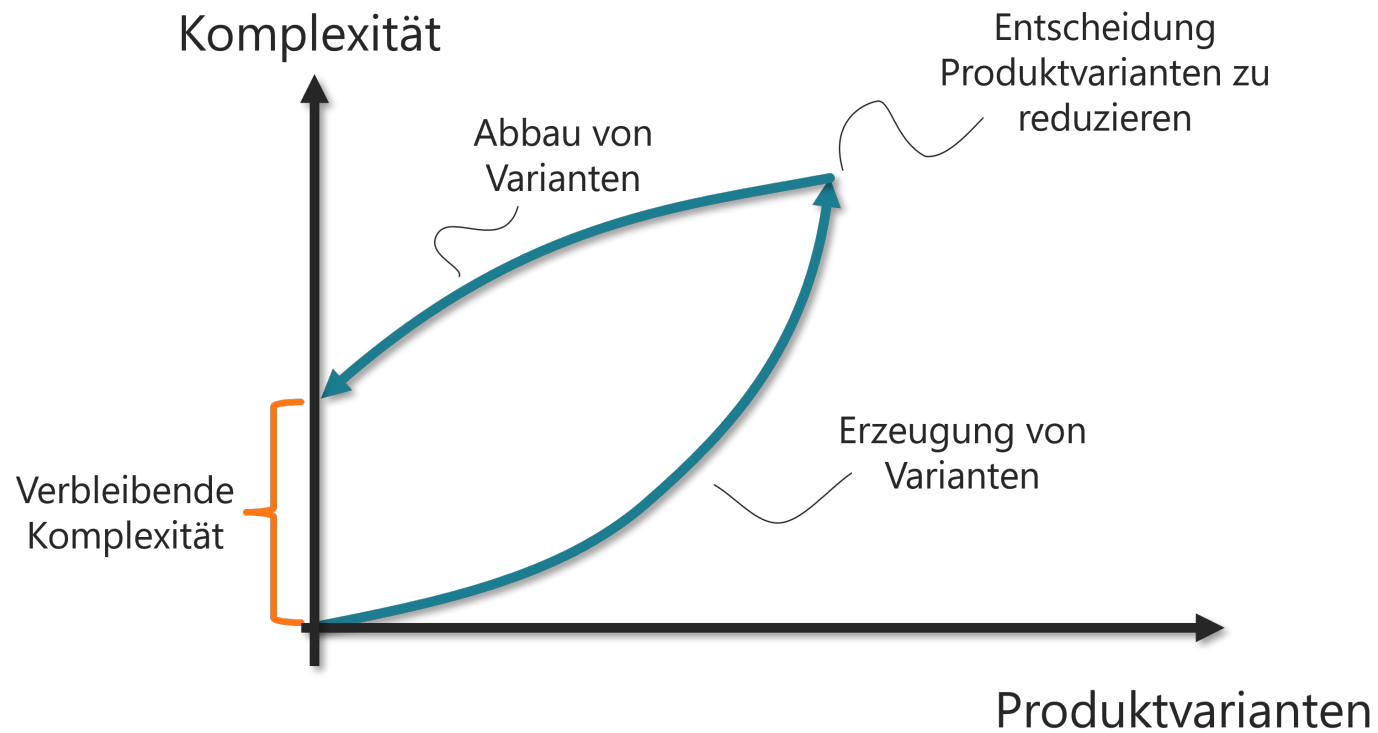
Management von Varianten

Herausforderungen im Variantenmanagement

Herausforderung	Ursache
Komplexität durch viele Varianten	Zu viele Varianten und Abhängigkeiten
Redundanzen in Anforderungen	Ähnliche Anforderungen für verschiedene Varianten
Inkonsistenzen zwischen Varianten	Manuelle Pflege von Varianten
Aufwand für Pflege	Manuelle Aktualisierung von Varianten
Schwierige Testbarkeit	Varianten-spezifische Testfälle
Fehlende Akzeptanz im Team	Unklarer Nutzen von Variantenmanagement
Compliance-Risiken	Unterschiedliche Vorgaben pro Variante

Management von Varianten

Komplexität durch Variantenmanagement



Quelle: <https://www.modularmanagement.com/de/blog/agile-lean-und-modularisierung>

Management von Varianten

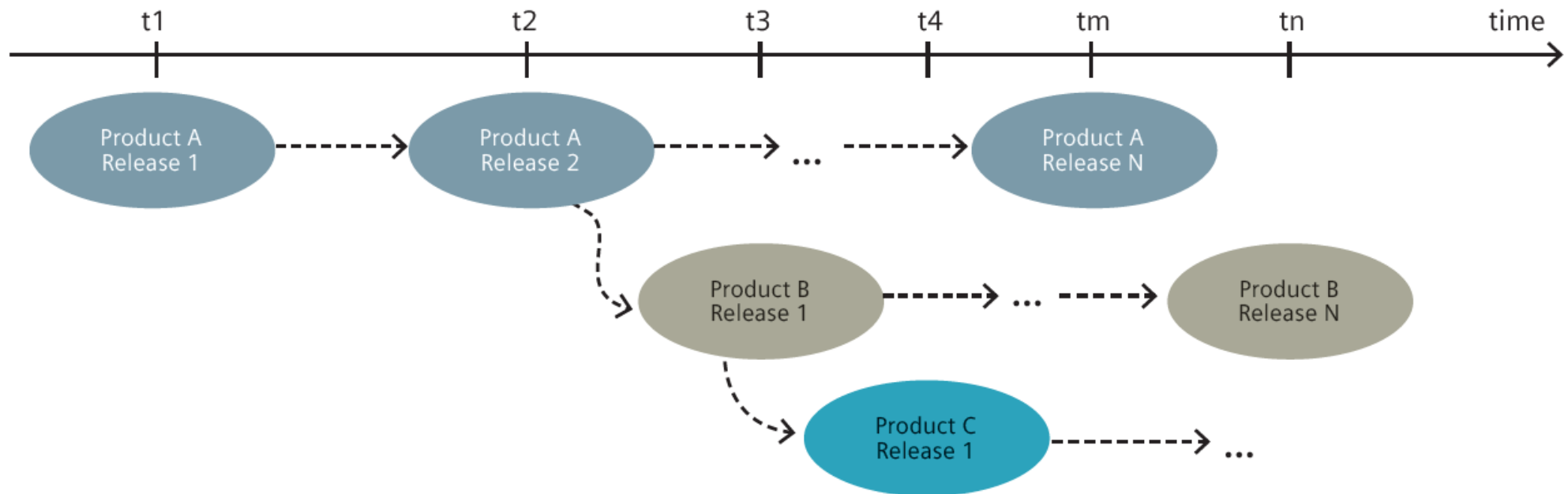
Varianten von Produkten

Produkt-Varianten können unterschiedlich identifiziert werden:

- Zuordnung zu bestimmten Varianten-Merkmalen
 - Merkmal = sichtbare Eigenschaft oder Qualität eines Systems
- Zuordnung zur konkreten Variante = zum konkreten Produkt
- **Techniken:**
 - textuell,
 - mit Attributen,
 - mit Tabellen,
 - mit Merkmalsmodellen

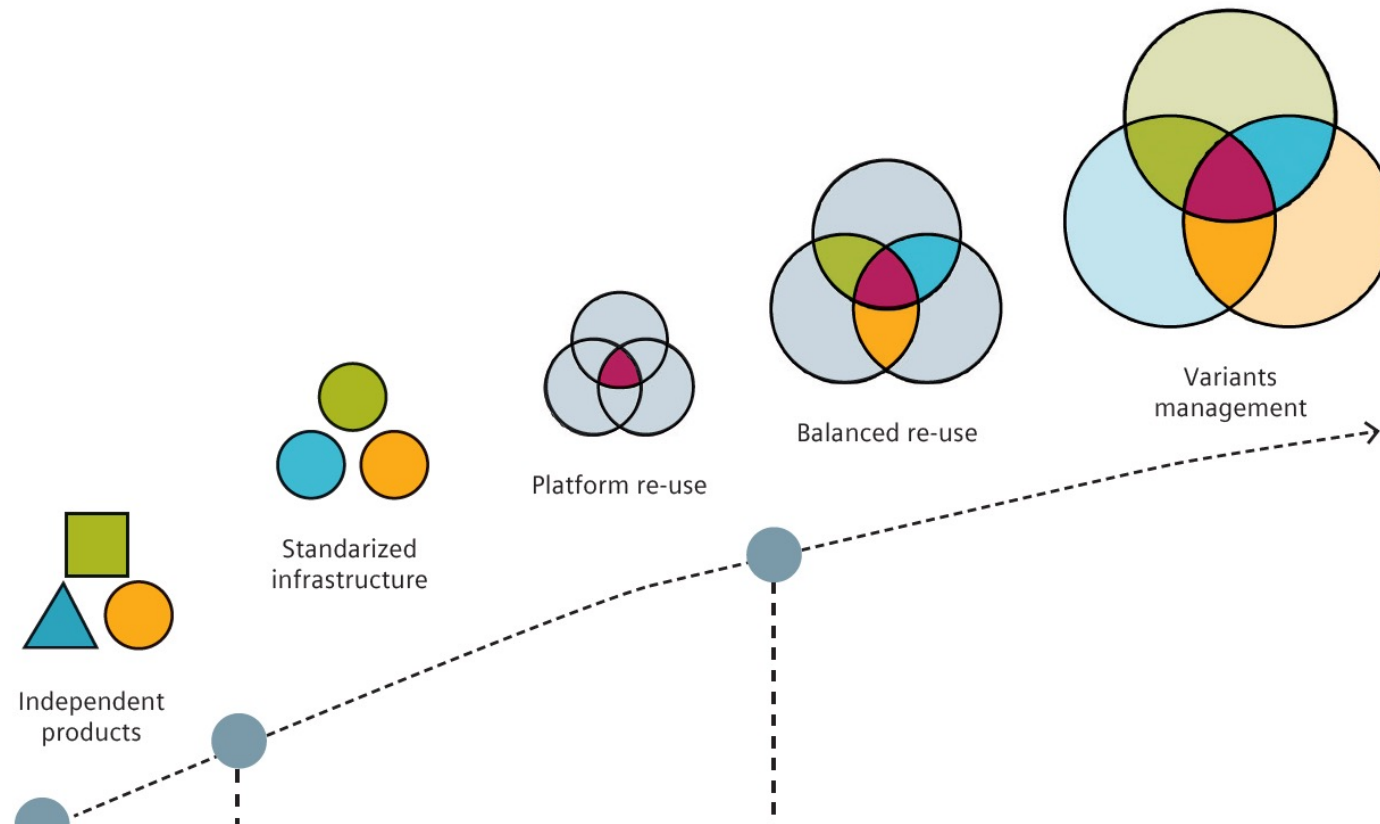
Management von Varianten

Entstehung von Varianten



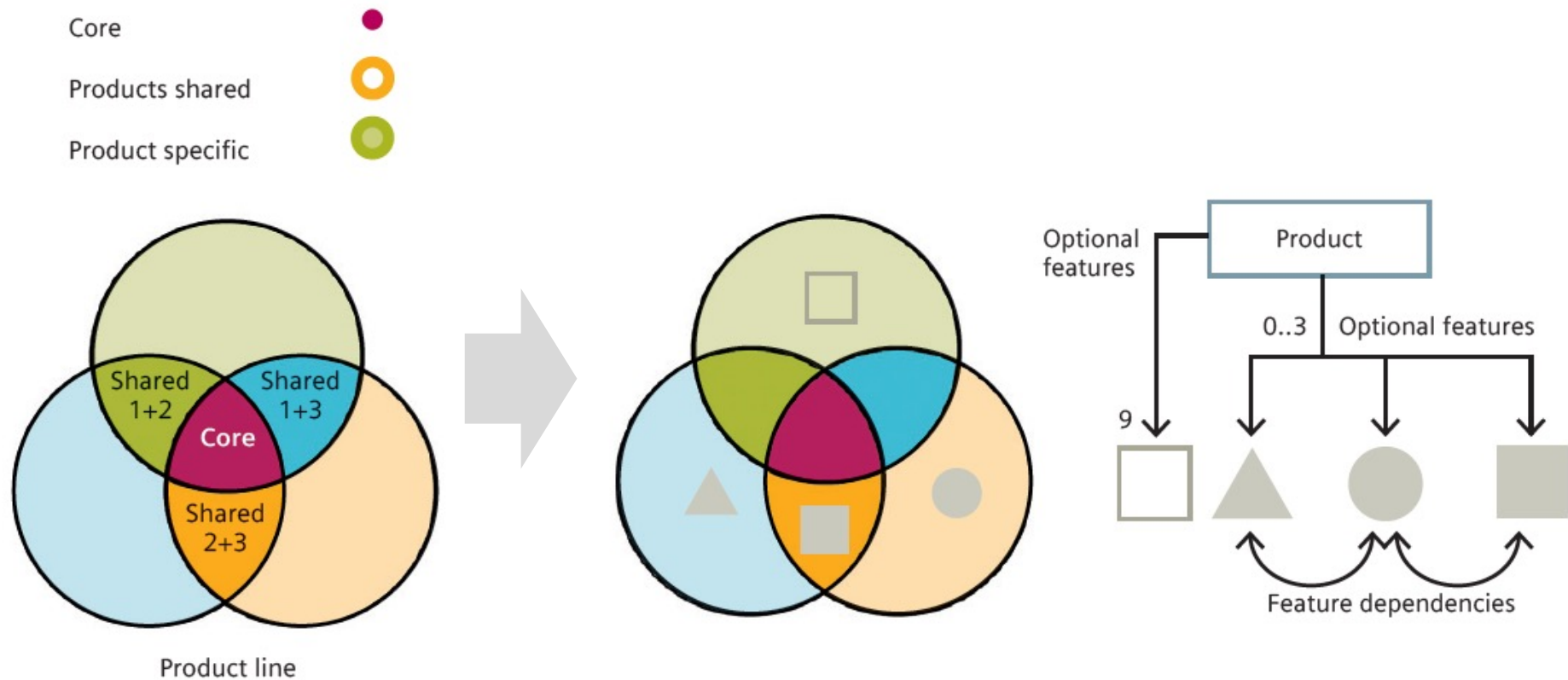
Management von Varianten

Arten von Varianten



Management von Varianten

Prinzip des Variantenmanagements



Management von Varianten

Zuordnung von Anforderungen zu Produkten

Anf-ID	Anforderung	Status	...
Flitz_0045	Der Parkassistent des Flitzers mit der Ausstattung Standard soll den Fahrer mit Hilfe von akustischen Signalen, die den Abstand zu Hindernissen anzeigen, beim Einparkvorgang unterstützen.	hoch	...
Flitz_0046	Der Parkassistent des Flitzers mit der Ausstattung Premium soll die Lenkung des Wagens während des Einparkens steuern.	hoch	...

Management von Varianten

Zuordnung von Anforderungen zu Produkten

Anf-ID	Anforderung	Standard-Ausstattung	Premium-Ausstattung	Status
Flitz_0055	Der Parkassistent des Flitzers soll den Fahrer mit Hilfe von akustischen Signalen, die den Abstand zu Hindernissen anzeigen, beim Einparkvorgang unterstützen.	X	-	hoch
Flitz_0056	Der Parkassistent des Flitzers soll die Lenkung des Wagens während des Einparkens steuern.	-	X	hoch

Management von Varianten

Zuordnung von Anforderungen zu Produkten

Anf-ID	Anforderung	Kundengruppe	Karosserie	Status
Flitz_0724	Der Flitzer soll bei schönem Wetter gefahren werden.	Freizeitfahrer	Cabrio	hoch
Flitz_0725	Der Flitzer soll bei Regenwetter gefahren werden können ohne dass der Fahrer nass wird.	Pendler	Kompakt, Limousine, Kombi	hoch
Flitz_0726	Der Flitzer soll ausreichend Platz zum Transport sperriger Teile bieten.	Unternehmer	Kombi	hoch
Flitz_0727	Der Flitzer soll dem Fahrer und bis zu vier Mitfahrern auch auf längeren Strecken einen angenehmen Komfort bieten.	Pendler, Reisende, Außendienstler	Limousine, Kombi	hoch

Management von Varianten

Gruppenaufgabe

- Erstellen Sie Varianten für unseren smarten Mähdrescher und ordnen Sie Anforderungen zu diesen Varianten zu.
- Nutzen Sie exemplarisch eine Tabelle mit ID wie auf den vorhergehenden Folien.
- Zeiteinsatz: 15 Minuten

Literatur

- Chris Rupp (2014): Requirements-Engineering und –Management, 6. Auflage, Hanser, München.
- BMI (2013): V-Modell XT Bund, https://download.gsb.bund.de/BundesCIO/V-Modell_XT_Bund/V-Modell-XT-Bund-1.1-Gesamt.pdf
- Dorsch (2025), Skript Requirements Engineering - Tag 1
- Dorsch (2025b), Skript Requirements Engineering - Tag 2
- Dorsch (2025c), Skript Requirements Engineering - Tag 3